Idenso

Symbolic tensor identities for Spenso and Symbolica

---

1 Overview

Idenso is the symbolic identity and simplification layer for tensors encoded in the form that Spenso can parse from Symbolica expressions. It provides representation symbols, index tooling, reversible "cooking" of subexpressions, selective expansion, and identities for Dirac, metric, epsilon, and color algebra.

> **Symbolic, not component expansion**
> Idenso rewrites abstract tensor expressions. Functions such as `expand_mink` distribute factorized terms that carry selected index families; they do not turn every tensor into a dense array of components. Concrete tensor storage and network execution belong to Spenso.

1.1 Choose a task

- To initialize representations and verify one metric contraction, follow the controlled identity tutorial and the Python function reference.
- To isolate dummy-index namespaces or cook a large expression, use the algebra guide with the exact `IndexTooling` and `Cookable` references.
- To audit a sign or normalization in a Dirac/color pass, use the source-backed FORM, Symbolica, and Spenso rule specification.

1.2 A controlled rewrite pipeline

There is no universally correct "simplify everything" order. A robust workflow makes each phase explicit:

- initialize the standard representations and tensor symbols;
- inspect dangling indices and normalize or wrap dummy-index namespaces when combining expressions;
- expand only the sectors needed by the next identity pass;
- simplify gamma, metric, or color structures as appropriate;
- canonicalize and compare results using the conventions of the consuming calculation.

Expansion can grow an expression quickly, so perform it as late and selectively as possible. Wrapping indices is especially important before multiplying independently constructed expressions: equal printed index names can otherwise acquire an unintended contraction.

1.3 Representations and cooking

The representation layer defines spin-fundamental, color-fundamental, color-sextet, bispinor, and color-adjoint types together with their duality. `initialize()` forces the related Symbolica symbols and Spenso tags to be registered before parsing or rewriting expressions.

The `Cookable` API can replace selected function-like subexpressions or index payloads with compact symbols and later reverse the operation. Use reversible settings when the original expression must be recovered. A cooked symbol is an intermediate encoding, not a portable serialized physics result unless the associated mapping and settings are retained.

Idenso is consumed by GammaLoop and its tensor conventions are shared with Spenso. Continue to the interface guide and generated Rust/Python references for supported functions, signatures, and feature gates.

Contributors changing representation syntax, symbolic-network parsing, index ownership, or rewrite order should also read the source-audited Idenso implementation architecture.

2 Tutorial

This tutorial uses Idenso's Python community module to contract one metric tensor with a vector. It is intentionally small: the point is to establish the registered tensor syntax and observe one algebra pass before composing a larger Dirac or color pipeline.

2.1 Prerequisites

Idenso is mounted at `symbolica.community.idenso`; it is not installed by `pip install idenso`. Use a Symbolica Python distribution whose release includes the Idenso and Spenso community modules, or build those modules through the `symbolica-community` assembly. Verify the actual environment first:

The assembly and installation instructions live in `symbolica-community`.

```
python -c "import symbolica.community.idenso; import symbolica.community.spenso"
```

If that fails, install or rebuild the community assembly before continuing. Generating a `.pyi` file does not mount the native module.

2.2 Simplify one metric contraction

Save the following as `metric_first.py`:

```
from symbolica.community.idenso import initialize, list_dangling, simplify_metrics
from symbolica.community.spenso import Representation, TensorName

initialize()
rep = Representation.euc(3)
mu = rep("mu")
nu = rep("nu")

g = TensorName.g()
q = TensorName("q")
expression = g(mu, nu) * q(mu)

print("free before:", list_dangling(expression))
reduced = simplify_metrics(expression)
print("reduced:", reduced)
print("free after:", list_dangling(reduced))
```

Run `python metric_first.py`. Success means the metric is removed from the reduced expression, the contracted `mu` no longer appears as a free index, and the result is a rank-one expression carrying `nu`. Printed output can vary with the installed Symbolica version; inspect the expression structure instead of comparing exact text.

> **Verification scope and cost**
> The docs harness compiles this Python source without importing native modules; it syntax-checks the environment probe without running it. Execute both steps in a provisioned community-module environment to verify the rank-one invariant. The script should take seconds after startup; building or installing Symbolica community modules is the expensive prerequisite and may take substantially longer.

> **Initialize before parsing or rewriting**
> `initialize()` installs Idenso's representation and tensor symbols. Calling it explicitly at the start of a

> standalone script makes registration order clear and keeps expressions in the Spenso-compatible form that Idenso's matchers expect.

2.3 Grow this into an algebra pipeline

Keep transformations observable while developing:

- call `list_dangling` before and after a pass to catch accidental contractions;
- use `wrap_dummies` before multiplying expressions built in independent index namespaces;
- expand only the Minkowski, bispinor, or color sector needed by the next pass;
- apply `simplify_metrics`, `simplify_gamma`, and `simplify_color` as distinct phases;
- canonicalize only after the physics-specific identities required by the calculation are explicit.

2.4 Troubleshooting and next steps

- `ModuleNotFoundError` for `symbolica.community.idenso` means the installed Symbolica build does not include the native module; a `.pyi` type stub or source checkout alone is not enough.
- If the metric remains unchanged, construct its slots with the same `Representation` and abstract index objects as the vector. Plain Symbolica functions do not automatically carry Spenso tensor structure.
- If a free index disappears unexpectedly, inspect repeated names and use `wrap_dummies` before combining separately constructed expressions.
- Continue with the syntax-and-algebra manual, then use the Python and Rust API references for signatures and feature gates.

3 Syntax, indices, and algebra passes

Idenso consumes the Symbolica function form emitted for Spenso structures. Function heads carry representation meaning and their arguments carry abstract indices or tensor data. Initialize Idenso before parsing so Symbolica attributes, Spenso tags, and dual representations are all registered in one deterministic order.

3.1 Indices and cooking

Free indices describe the result; repeated compatible indices describe contractions. Before combining independently built expressions, wrap or rename dummy-index namespaces so equal printed names do not create accidental contractions. Cooking temporarily replaces selected index payloads or function subexpressions with compact symbols. Retain the generated map and settings whenever uncooking is required.

Cooking is especially useful before expensive canonicalization, but it changes what downstream matchers can see. Uncook before applying an identity whose pattern depends on the hidden tensor head or index structure.

3.2 Keep independent dummy namespaces independent

Two factors may legitimately print the same local dummy name before they are multiplied. Wrap each factor with a distinct header first, while leaving its external indices untouched:

```
import symbolica as sp
from symbolica.community.idenso import initialize, list_dangling, wrap_dummies
from symbolica.community.spenso import Representation, TensorName

initialize()
rep = Representation.euc(3)
mu = rep("mu")
nu = rep("nu")
rho = rep("rho")
g = TensorName.g()
p = TensorName("p")
```

```
q = TensorName("q")

left = g(mu, nu) * p(mu)
right = g(mu, rho) * q(mu)
safe_product = wrap_dummies(left, sp.S("lhs")) * wrap_dummies(right, sp.S("rhs"))

assert len(list_dangling(safe_product)) == 2
```

The stable invariant is two free indices, `nu` and `rho`; the two occurrences of local `mu` belong to separate contractions after wrapping. The generated `wrap_dummies` reference records the Python signature, while the exact `IndexTooling` reference covers the underlying Rust boundary.

> **Interpret index failures before simplifying**
> More or fewer than two dangling indices means a name collided or a slot's representation or duality differs from the intended one. An unchanged plain Symbolica function means it was not constructed through Spenso tensor names/representations, or `initialize()` ran after parsing. Correct those structural issues before metric, Dirac, or color simplification.

3.3 Metric and epsilon operations

Metric contraction raises, lowers, or identifies compatible Lorentz indices according to the registered representation. Epsilon identities depend on dimension, ordering, and sign conventions; expand them only when the next pass benefits from the larger expression. Keep Minkowski expansion selective to avoid distributing unrelated scalar factors.

3.4 Dirac and color algebra

Dirac passes simplify gamma chains, traces, slashes, and spinor-compatible contractions. Color passes handle fundamental/adjoint deltas, generators, structure constants, and registered group parameters. Apply one algebra family at a time and inspect the intermediate expression; an all-at-once fixed-point loop can obscure which convention produced a sign or normalization.

> **Canonical does not mean physically equivalent by itself**
> Canonical ordering makes structurally equivalent expressions comparable under the registered rules. On-shell relations, gauge choices, dimension-specific identities, and model parameter substitutions must still be requested explicitly.

3.5 Schoonschip network parsing

The Schoonschip path parses large gamma-chain expressions into the same Spenso-compatible network model before contraction. It resolves aliases and normalizes the expression before constructing the network. Spenso then plans and executes contractions, while Idenso supplies the representation and algebra rules. Avoid substituting scalar parameters too early: doing so can substantially increase intermediate expression size even when the resulting tensor network is unchanged.

Concrete syntax and rewrite cases are documented in the Spenso/Symbolica syntax note and the rendered FORM color and Dirac specification. The Schoonschip parsing guide shows how normalization, network construction, and contraction fit together.

Tensor storage and execution remain owned by Spenso.

4 FORM color and Dirac rules

This specification records source-backed replacement rules for FORM gamma-algebra simplification and the FORM `color.h` color-algebra package. The mathematical notation is independent of FORM token names except

where a source symbol is documented literally. Open gamma strings always carry explicit bispinor indices; closed gamma traces are written as explicit bispinor contractions, not as a separate trace-index placeholder.

The Symbolica + spenso section uses the approved `chain` and `trace` tensor-pattern notation:

```
chain(bis(4,a),bis(4,c),
  gamma(in,out,p(2,mink(4))),
  gamma(in,out,mink(4,mu)))
```

and

```
chain(cof(Nc,i),dind(cof(Nc,j)),
  t(coad(Nc^2-1,a),in,out),
  t(coad(Nc^2-1,b),in,out))
```

This is a specification-level pattern notation. GammaLoop currently has internal helpers named `spenso::gamma_chain` and `spenso::gamma_trace`; those names are mentioned only when documenting the implementation source.

4.1 Source Inventory

- FORM manual gamma-algebra section: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.
- FORM current source repository, gamma trace implementation: https://github.com/form-dev/form/blob/master/sources/opera.c.
- FORM package page for `color.h`: https://www.nikhef.nl/~form/maindir/packages/color/color.html.
- FORM package source `color.h`: https://www.nikhef.nl/~form/maindir/packages/color/color.h. This source is not currently present in the `form-dev/form` GitHub tree; line references to `color.h` therefore use package-source line numbers recorded in this document and not GitHub anchors.
- FORM color example programs: https://www.nikhef.nl/~form/maindir/packages/color/color.tar.gz. The package page describes these as simple FORM programs that use `color.h`.
- GammaLoop/idenso current tensor simplifiers: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs, https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/color/mod.rs.
- Symbolica pattern matching documentation: https://symbolica.io/docs/pattern_matching.html.
- Symbolica Rust `id` API documentation: https://docs.rs/symbolica/latest/symbolica/id/index.html, https://docs.rs/symbolica/latest/symbolica/id/struct.ReplaceBuilder.html.
- spenso Rust crate documentation: https://docs.rs/spenso/latest/spenso/, symbolic tensor-library source: https://docs.rs/spenso/latest/src/spenso/network/library/symbolic.rs.html.
- Typst syntax and math documentation: https://typst.app/docs/reference/syntax/, https://typst.app/docs/reference/math/.

4.2 Notational Conventions

Lorentz indices are written as μ, ν, ρ, σ and are raised and lowered with $\eta^{\mu\nu}$ and $\eta_{\mu\nu}$. The dimension is denoted by D in dimension-generic identities and fixed to 4 in the four-dimensional FORM `trace4` rules. Repeated Lorentz labels are contracted.

Bispinor indices are $\alpha, \beta, \chi, \delta$. An open gamma chain is written as $(\Gamma^{\mu_1 \cdots \mu_n})^{\beta}_{\alpha}$. A closed trace is an explicit contraction $(\Gamma^{\mu_1 \cdots \mu_n})^{\alpha}_{\alpha}$.

Fundamental color indices are i, j, k, l. Adjoint color indices are a, b, c, d, e. Fundamental generators are $(T^a)^{j}_{i}$. The package constants are mapped as $C_A \leftrightarrow$ `cA`, $C_F \leftrightarrow$ `cR`, $T_R \leftrightarrow$ `I2R`, and $N_c \leftrightarrow$ `NR`; `NA` is the adjoint dimension. For ordinary $\mathrm{SU}(N_c)$, $C_A = N_c$, $T_R = \frac{1}{2}$, and $C_F = \frac{N_c^2 - 1}{2N_c}$, but `color.h` is written in terms of invariants.

The word `product` denotes an ordered noncommutative string inside a chain. It is not used as FORM syntax.

4.3 Gamma-algebra replacement rules

4.3.1 Overview

Rule	Dimension	Purpose
Gamma collection	all	Collect adjacent bispinor-contracted gamma factors into an ordered chain.
Chain joining	all	Join two open chains when the outgoing bispinor index of one equals the incoming index of the next.
Metric contraction	all or D	Replace $\gamma^\mu\gamma_\mu$ and related repeated Lorentz slots by the active dimension.
Trace closure	all	Turn an open chain with equal endpoints into an explicit bispinor contraction.
Trace recursion	D or 4	Evaluate even traces recursively and annihilate odd traces without gamma-five insertions.
Chisholm reduction	4 only	Reduce $\gamma^\mu\Gamma\gamma_\mu$ using four-dimensional identities.
Gamma-five and epsilon	4 only	Use Levi-Civita and gamma-five branch logic in FORM `trace4`.

4.3.2 G1. Gamma-chain product and joining

Mathematical identity:

$$\gamma(v)^\beta_\alpha\gamma(w)^\chi_\beta = (\Gamma^{vw})^\chi_\alpha \tag{25}$$

For longer strings, the same rule constructs an ordered product inside the chain:

$$(\Gamma^{v_1\dots v_m})^\beta_\alpha(\Gamma^{w_1\dots w_n})^\chi_\beta = (\Gamma^{v_1\dots v_m w_1\dots w_n})^\chi_\alpha \tag{26}$$

FORM source context:

```
/* Trace4 receives a string of gamma matrices in 'instring'. */
number = *params - 6;
...
for ( i = 0; i < number; i++ ) *p++ = *m++;
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L670-L686, https://github.com/form-dev/form/blob/master/sources/opera.c#L715-L745. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

GammaLoop/idenso implementation currently collects adjacent gamma matrices by replacing two contracted `spenso::gamma` calls with an internal chain:

```
AGS.gamma_pattern(RS.a__, RS.b__, RS.c__) * AGS.gamma_pattern(RS.d__, RS.c__, RS.e__),
GS.chain_pattern(RS.b__, RS.e__, [RS.a__, RS.c__, RS.d__]),
```

Source: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L604-L613.

Symbolica + spenso pattern:

```
gamma(bis(4,a),bis(4,b),p(2,mink(4)))
* gamma(bis(4,b),bis(4,c),mink(4,mu))
  -> chain(bis(4,a),bis(4,c),
       gamma(in,out,p(2,mink(4))),
       gamma(in,out,mink(4,mu)))
```

Assumptions: `in` and `out` are local slots of each factor in the chain. Dummy bispinor labels consumed during collection are not retained as free indices.

4.3.3 G2. Chain orientation and reversal

Mathematical identity:

$$(\Gamma^{v_1\dots v_n})^{\beta}_{\alpha} = \left((\Gamma^{v_n\dots v_1})^{\alpha}_{\beta}\right)^{\mathrm{rev}} \tag{27}$$

Here `rev` means that the implementation may represent the same contracted tensor network with opposite local slot orientation. This is not a gamma-algebra identity by itself; it is a tensor-pattern canonicalization convention.

Pattern examples:

```
A(1,j,la,i,mu) * B(j,k) * C(k,l)
  -> chain(rep,i,l,A(1,out,la,in,mu),B(in,out),C(in,out))

B(j,k) * C(l,k)
  -> chain(rep,j,l,B(in,out),C(out,in))
```

Source context: Symbolica matches syntactically with wildcard variables and respects function-argument ordering for non-symmetric functions; see https://symbolica.io/docs/pattern_matching.html. spenso provides symbolic tensor structures and contraction support; see https://docs.rs/spenso/latest/spenso/.

4.3.4 G3. Metric contraction in gamma chains

Dimension-generic terminal identity:

$$\gamma^{\mu}_{\alpha\beta}\gamma^{\chi}_{\mu,\beta} = D\delta^{\chi}_{\alpha} \tag{28}$$

In the approved chain notation:

```
chain(bis(D,a),bis(D,c),
  gamma(in,out,mink(D,mu)),
  gamma(in,out,mink(D,mu)))
  -> D * g(bis(D,a),bis(D,c))
```

FORM source context: `Trace4Gen` tests repeated adjacent objects and puts the repeated pair into the accumulated delta list before recursing.

```
if ( *p == p[1] ) {
  *(t->accup)++ = *p;
  *(t->accup)++ = *p;
  ...
  Trace4Gen(..., number-2)
}
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L958-L972. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

GammaLoop/idenso records the dimension contraction in its repeated-Lorentz chain patterns:

```
function!(GS.gamma_chain, ..., mink(d_,a_), ..., mink(d_,a_), ...)
  -> function!(GS.gamma_chain, ...) * d_
```

Source: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L784-L829.

Assumptions: D is the dimension attached to the Lorentz representation. Four-dimensional Chisholm rules below require $D = 4$.

4.3.5 G4. Trace closure with explicit bispinor contraction

Mathematical identity:

$$(\Gamma^{v_1\cdots v_n})^{\alpha}_{\alpha} = \gamma(v_1)^{\beta}_{\alpha}\ldots\gamma(v_n)^{\alpha}_{\chi} \tag{29}$$

Pattern:

```
chain(bis(4,a),bis(4,a),
  gamma(in,out,p(2,mink(4))),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,p(3,mink(4))))
  -> trace(bis(4),
       gamma(in,out,p(2,mink(4))),
       gamma(in,out,mink(4,mu)),
       gamma(in,out,p(3,mink(4))))
```

FORM source context: FORM `trace4` and `tracen` operate on a spin line, represented internally as a string of gamma matrices. `Trace4` copies that string into `t->inlist` and then dispatches to `Trace4Gen`.

```
t->inlist = AT.WorkPointer;
...
for ( i = 0; i < number; i++ ) *p++ = *m++;
...
ret = Trace4Gen(...);
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L730-L745, https://github.com/form-dev/form/blob/master/sources/opera.c#L820-L823. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

GammaLoop/idenso closes chains when the endpoints agree:

```
replace(function!(GS.gamma_chain, RS.a__, RS.x_, RS.x_).to_pattern())
  .repeat()
  .with(function!(GS.gamma_trace, RS.a__).to_pattern())
```

Source: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L877-L879.

4.3.6 G5. Odd and even trace recursion

Odd trace without gamma-five:

$$(\Gamma^{v_1\cdots v_{2k+1}})^{\alpha}_{\alpha} = 0 \tag{30}$$

Two-gamma trace:

$$\gamma^{\mu}_{\alpha\beta}\gamma^{\nu}_{\beta\alpha} = 4\eta^{\mu\nu} \tag{31}$$

Recursive even trace:

$$(\Gamma^{v_1\cdots v_{2k}})^{\alpha}_{\alpha} = \sum_{r=2}^{2k} (-1)^r \eta^{v_1 v_r} \left(\Gamma^{v_2\cdots\widehat{v_r}\cdots v_{2k}}\right)^{\beta}_{\beta} \tag{32}$$

FORM source context:

```
if ( ( number < 0 ) || ( number & 1 ) ) return(0);
...
if ( number == 2 ) { *kron++ = *t->inlist; *kron++ = t->inlist[1]; }
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432. The recursive trace generator comments describe zero- and two-gamma terminal cases before general recursion: https://github.com/form-dev/form/blob/master/sources/opera.c#L830-L856. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

Symbolica + spenso pattern:

```
trace(bis(4),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)))
  -> 4 * g(mink(4,mu),mink(4,nu))
```

Dummy-index freshness: recursive trace replacement removes the paired Lorentz argument and must not reuse bound labels introduced by metric contractions.

4.3.7 G6. Four-dimensional Chisholm reductions

Odd interior:

$$\gamma^{\mu}_{\alpha\beta}(\Gamma^{\nu_1\cdots\nu_{2k+1}})^{\chi}_{\beta}\gamma^{\delta}_{\mu,\chi} = -2(\Gamma^{\nu_{2k+1}\cdots\nu_1})^{\delta}_{\alpha} \tag{33}$$

Even interior:

$$\gamma^{\mu}_{\alpha\beta}(\Gamma^{\nu_1\cdots\nu_{2k}})^{\chi}_{\beta}\gamma^{\delta}_{\mu,\chi} = 2(\Gamma^{\nu_{2k}\nu_1\cdots\nu_{2k-1}})^{\delta}_{\alpha} + 2(\Gamma^{\nu_{2k-1}\cdots\nu_1\nu_{2k}})^{\delta}_{\alpha} \tag{34}$$

Special two-interior case:

$$\gamma^{\mu}_{\alpha\beta}\gamma^{\nu}_{\beta\chi}\gamma^{\rho}_{\chi\lambda}\gamma^{\delta}_{\mu,\lambda} = 4\eta^{\nu\rho}\delta^{\delta}_{\alpha} \tag{35}$$

FORM source comment:

```
g(mu)*g(a1)*...*g(an)*g(mu)=
n=odd:  -2*g(an)*...*g(a1)
n=even: 2*g(an)*g(a1)*...*g(a(n-1))
    +2*g(a(n-1))*...*g(a1)*g(an)
There is a special case for n=2 : 4*d(a1,a2)*gi
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L670-L680. Implementation odd contraction: https://github.com/form-dev/form/blob/master/sources/opera.c#L978-L1012. Implementation even contraction: https://github.com/form-dev/form/blob/master/sources/opera.c#L1021-L1096, https://github.com/form-dev/form/blob/master/sources/opera.c#L1118-L1157. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

Pattern:

```
chain(bis(4,a),bis(4,d),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu1)),
  gamma(in,out,mink(4,nu2)),
```

```
  gamma(in,out,mink(4,nu3)),
  gamma(in,out,mink(4,mu)))
  -> -2 * chain(bis(4,a),bis(4,d),
       gamma(in,out,mink(4,nu3)),
       gamma(in,out,mink(4,nu2)),
       gamma(in,out,mink(4,nu1)))
```

Assumptions: The contracted Lorentz label must belong to a four-dimensional Lorentz representation. The source code checks dimension equality with 4 before applying the Chisholm branches.

4.3.8 G7. Gamma-five and epsilon rules

FORM `trace4` uses the four-dimensional identity:

$$\gamma^\mu\gamma^\nu\gamma^\rho = \varepsilon^{\mu\nu\rho\sigma}\gamma_5\gamma_\sigma + \eta^{\mu\nu}\gamma^\rho - \eta^{\mu\rho}\gamma^\nu + \eta^{\nu\rho}\gamma^\mu \tag{36}$$

Source comment:

```
g_(j,mu)*g_(j,nu)*g_(j,ro)=e_(mu,nu,ro,si)*g5_(j)*g_(j,si)
+d_(mu,nu)*g_(j,ro)-d_(mu,ro)*g_(j,nu)+d_(nu,ro)*g_(j,mu)
which is for 4 dimensions only!
```

Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L320-L329. The gamma-five branch logic in trace generation is at https://github.com/form-dev/form/blob/master/sources/opera.c#L480-L489 and https://github.com/form-dev/form/blob/master/sources/opera.c#L808-L813. Manual: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.

Pattern:

```
chain(bis(4,a),bis(4,b),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)),
  gamma(in,out,mink(4,rho)))
  -> epsilon(mink(4,mu),mink(4,nu),mink(4,rho),mink(4,sigma))
     * chain(bis(4,a),bis(4,b),
         gamma5(in,out),
         gamma(in,out,mink(4,sigma)))
     + g(mink(4,mu),mink(4,nu)) * chain(... gamma(rho) ...)
     - g(mink(4,mu),mink(4,rho)) * chain(... gamma(nu) ...)
     + g(mink(4,nu),mink(4,rho)) * chain(... gamma(mu) ...)
```

This is explicitly four-dimensional and requires a fresh dummy Lorentz label `sigma`.

4.4 Color-algebra replacement rules from `color.h`

4.4.1 Overview

Rule	Convention	Purpose
Line joining	`T(i,j,?a)`	Join open noncommutative color lines.
Trace closure	`cOlTr`, `cOlTt`	Convert closed lines to trace blocks and choose trace blocks to simplify.
Casimir reduction	`cR`, `cA`	Remove repeated adjacent or separated generators in traces.
Trace normalization	`I2R`	Evaluate two-generator trace.

Rule	Convention	Purpose
Structure constants	`f` antisymmetric	Contract small f loops and rewrite larger loops through invariants.
Symmetric invariants	`cOldR`, `cOldA`	Represent and contract generalized d tensors.
`simpli` strategy	procedure	Eliminate f tensors in environments of invariants using generalized Jacobi moves.

4.4.2 C1. Open color-line joining

Mathematical identity:

$$\left(T^{a_1}\ldots T^{a_m}\right)_i^j\left(T^{b_1}\ldots T^{b_n}\right)_j^k = \left(T^{a_1}\ldots T^{a_m}T^{b_1}\ldots T^{b_n}\right)_i^k \tag{37}$$

FORM source:

```
repeat id T(cOli1?,cOli2?,?a)*T(cOli2?,cOli3?,?b) = T(cOli1,cOli3,?a,?b);
id  T(cOli1?,cOli1?,?a) = cOlTr(?a);
```

Source package lines: `color.h` lines 91–93 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html. No official GitHub line anchor was found for `color.h`; see the validation appendix.

Pattern:

```
chain(cof(Nc,i),dind(cof(Nc,j)),
  t(coad(Nc^2-1,a1),in,out),
  t(coad(Nc^2-1,a2),in,out))
*
chain(cof(Nc,j),dind(cof(Nc,k)),
  t(coad(Nc^2-1,b1),in,out))
  -> chain(cof(Nc,i),dind(cof(Nc,k)),
       t(coad(Nc^2-1,a1),in,out),
       t(coad(Nc^2-1,a2),in,out),
       t(coad(Nc^2-1,b1),in,out))
```

4.4.3 C2. Trace closure and trace normalization

Closed line:

$$\left(T^{a_1}\ldots T^{a_n}\right)_i^i = \mathrm{tr}_R\left(T^{a_1}\ldots T^{a_n}\right) \tag{38}$$

One generator:

$$\mathrm{tr}_R\left(T^a\right) = 0 \tag{39}$$

Two generators:

$$\mathrm{tr}_R\left(T^aT^b\right) = T_R\delta^{ab} \tag{40}$$

FORM source:

```
id  cOlTt(cOli1?) = 0;
id  cOlTt(cOli1?,cOli2?) = I2R*d_(cOli1,cOli2);
id  cOlTt(cOli1?,cOli2?,cOli3?) = cOldR(...) + i_/2*f(...)*I2R;
```

Source package lines: `color.h` lines 459–461 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern:

```
chain(cof(Nc,i),dind(cof(Nc,i)),
  t(coad(Nc^2-1,a),in,out),
  t(coad(Nc^2-1,b),in,out))
  -> trace(cof(Nc),
       t(coad(Nc^2-1,a),in,out),
       t(coad(Nc^2-1,b),in,out))
  -> TR * g(coad(Nc^2-1,a),coad(Nc^2-1,b))
```

4.4.4 C3. Fundamental Casimir and adjacent contractions

Mathematical identities:

$$(T^a)^j_i (T^a)^k_j = C_F \delta^k_i \tag{41}$$

$$\mathrm{tr}_R\big(T^a T^b T^a \,\mathrm{product}(X)\big) = \left(C_F - \frac{C_A}{2}\right) \mathrm{tr}_R\big(T^b \,\mathrm{product}(X)\big) \tag{42}$$

FORM source:

```
id  cOlTr(cOli1?,cOli1?,?a) = cR*cOlTr(?a);
id  cOlTr(cOli1?,cOli2?,cOli1?,?a) = [cR-cA/2]*cOlTr(cOli2,?a);
```

Source package lines: `color.h` lines 108–111 at https://www.nikhef.nl/~form/maindir/packages/color/color.h, repeated for `cOlTt` at lines 173–175. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern:

```
chain(cof(Nc,i),dind(cof(Nc,k)),
  t(coad(Nc^2-1,a),in,out),
  t(coad(Nc^2-1,a),in,out))
  -> C_F * g(cof(Nc,i),dind(cof(Nc,k)))
```

Assumptions: `color.h` keeps `C_F` as `cR`, not automatically as $\frac{N_c^2-1}{2N_c}$.

4.4.5 C4. Fundamental Fierz generator contraction

Mathematical identity:

$$(T^a)^j_i (T^a)^l_k = T_R\left(\delta^l_i \delta^j_k - N_c^{-1} \delta^j_i \delta^l_k\right) \tag{43}$$

This identity is not written in this explicit form in `color.h`, whose main algorithm uses trace joining and invariant reductions. It is implemented directly in GammaLoop/idenso:

```
t(e,a,b) * t(e,c,d)
  -> TR * (id(a,d) * id(c,b) - id(a,b) * id(c,d) / Nc)
```

Source: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/color/mod.rs#L407-L414. Symbolica documentation for replacement mechanics: https://symbolica.io/docs/pattern_matching.html.

Pattern:

```
t(coad(Nc^2-1,a),cof(Nc,i),dind(cof(Nc,j)))
* t(coad(Nc^2-1,a),cof(Nc,k),dind(cof(Nc,l)))
  -> TR * (
       g(cof(Nc,i),dind(cof(Nc,l)))
       * g(cof(Nc,k),dind(cof(Nc,j)))
       - g(cof(Nc,i),dind(cof(Nc,j)))
       * g(cof(Nc,k),dind(cof(Nc,l))) / Nc)
```

4.4.6 C5. Structure-constant loop contractions

Mathematical identities:

$$f^{acd}f^{bcd} = C_A\delta^{ab} \tag{44}$$

$$f^{abe}f^{bcf}f^{cag} = \frac{C_A}{2}f^{efg} \tag{45}$$

FORM source:

```
ReplaceLoop,f,a=3,l<4,outfun=cOlff;
id  cOlff(cOli1?,cOli2?) = -cA*d_(cOli1,cOli2);
id  cOlff(cOli1?,cOli2?,cOli3?) = cA/2*f(cOli1,cOli2,cOli3);
```

Source package lines: `color.h` lines 148–150, 208–210, 505–507, and 538–541 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern:

```
f(coad(Nc^2-1,a),coad(Nc^2-1,c),coad(Nc^2-1,d))
* f(coad(Nc^2-1,b),coad(Nc^2-1,c),coad(Nc^2-1,d))
  -> ca * g(coad(Nc^2-1,a),coad(Nc^2-1,b))
```

Sign note: FORM's `ReplaceLoop` loop orientation produces `-cA*d_(...)` for the two-argument cyclic helper. The standard identity above assumes the displayed orientation $f^{acd}f^{bcd}$.

4.4.7 C6. Mixed trace–structure contraction

Mathematical identity:

$$\mathrm{tr}_R\big(T^aT^b\,\mathrm{product}(X)\big)f^{abc} = i\frac{C_A}{2}\,\mathrm{tr}_R(T^c\,\mathrm{product}(X)) \tag{46}$$

FORM source:

```
id  cOlTr(cOli1?,cOli2?,?a)*f(cOli1?,cOli2?,cOli3?) =
  i_*cA/2*cOlTr(cOli3,?a);
```

Source package lines: `color.h` lines 108–111 and 173–175 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern:

```
trace(cof(Nc),
  t(coad(Nc^2-1,a),in,out),
  t(coad(Nc^2-1,b),in,out),
  rest__)
* f(coad(Nc^2-1,a),coad(Nc^2-1,b),coad(Nc^2-1,c))
```

```
  -> i_ * ca / 2 * trace(cof(Nc),
       t(coad(Nc^2-1,c),in,out),
       rest__)
```

4.4.8 C7. Symmetric invariant contractions

`color.h` uses `cOldR` and `cOldA` for symmetric invariant tensors and reduces contractions to named invariants such as `d33`, `d44`, and `d55`.

Representative identities:

$$d_R^{abc} d_R^{abd} = d_{33}(R,R)\frac{\delta^{cd}}{N_A} \tag{47}$$

$$d_X^{abc} d_Y^{abc} = d_{33}(X,Y) \tag{48}$$

FORM source:

```
id cOldr1?cOldar[cOlx3](cOli1?,cOli2?,cOli3?)
 *cOldr2?cOldar[cOlx4](cOli1?,cOli2?,cOli4?)
 = d33(...)*cOlx1*d_(cOli3,cOli4)/NA;
...
id cOldr1?cOldar[cOlx3](cOli1?,...,cOli3?)
 *cOldr2?cOldar[cOlx4](cOli1?,...,cOli3?) = d33(...);
```

Source package lines: `color.h` lines 1287–1294 and 1316–1335 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern:

```
d(sym_rep_x,coad(Nc^2-1,a),coad(Nc^2-1,b),coad(Nc^2-1,c))
* d(sym_rep_y,coad(Nc^2-1,a),coad(Nc^2-1,b),coad(Nc^2-1,d))
  -> d33(sym_rep_x,sym_rep_y)
     * g(coad(Nc^2-1,c),coad(Nc^2-1,d)) / (Nc^2 - 1)
```

Assumptions: The package treats these as group-invariant placeholders rather than expanding them into N_c polynomials.

4.4.9 C8. `simpli` and generalized Jacobi strategy

`simpli` is an implementation strategy, not one scalar identity. It eliminates pairs of structure constants in environments of symmetric invariants and applies generalized Jacobi transformations.

FORM source:

```
* Procedure tries to eliminate f tensors in an environment of invariants.
* Apply the generalized Jacobi identity ...
if ( count(f,1) != multipleof(2) ) discard;
#call contract
...
id f(cOlpR`isimpli',cOli1?,cOli3?)*f(cOlpR`isimpli',cOli2?,cOli3?) =
  cOlpR`isimpli'(cOli1)*cOlpR`isimpli'(cOli2)*cA/2;
```

Source package lines: `color.h` lines 688–713 at https://www.nikhef.nl/~form/maindir/packages/color/color.h. Package documentation: https://www.nikhef.nl/~form/maindir/packages/color/color.html.

Pattern family:

```
f(coad(Nc^2-1,x),coad(Nc^2-1,a),coad(Nc^2-1,c))
* f(coad(Nc^2-1,x),coad(Nc^2-1,b),coad(Nc^2-1,c))
* invariant_environment__
  -> ca / 2 * projected_invariant(a,b,invariant_environment__)
```

This rule class requires canonicalization of symmetric invariants and freshness of all auxiliary adjoint labels introduced during generalized Jacobi expansion.

4.5 Symbolica and Spenso rewrite patterns

This section is written for a Rust implementation with Symbolica as a dependency. Symbolica's Rust `id` module exposes pattern matching through `AtomCore::replace`, `AtomCore::pattern_match`, `Replacement`, `MatchSettings`, and `ReplaceBuilder`; complex right-hand sides can be built with `ReplaceBuilder::with_map` or with `replace_map` callbacks. See https://docs.rs/symbolica/latest/symbolica/id/index.html and https://docs.rs/symbolica/latest/symbolica/id/struct.ReplaceBuilder.html. spenso represents tensor slots through symbolic structures and representation slots; the crate documents tensor structures, symbolic tensors, and contraction support at https://docs.rs/spenso/latest/spenso/, with symbolic tensor-library keys in https://docs.rs/spenso/latest/src/spenso/network/library/symbolic.rs.html.

Symbolica wildcard convention: `x_` is one atom, `x__` is one-or-more atoms, and `x___` is zero-or-more atoms. Function argument order is preserved for non-symmetric functions. Repeated wildcard names require exact structural equality; this is the right mechanism for repeated dummy labels such as `mu_` or `a_`.

The public target syntax for this specification is `chain(...)` and `trace(...)`. If implemented inside GammaLoop/idenso, the current internal Rust symbols are `spenso::gamma_chain` and `spenso::gamma_trace`; those should be adapter names only, not the public pattern vocabulary. The approved surface form remains:

```
chain(bis(4,a),bis(4,c),
  gamma(in,out,p(2,mink(4))),
  gamma(in,out,mink(4,mu)))
```

4.5.1 Rust pattern construction guidelines

Use concrete spenso builders for tensor slots where available and Symbolica wildcards only for the abstract labels. This keeps representation constraints attached to the pattern.

```
use symbolica::{
  atom::{Atom, AtomCore, AtomView, FunctionBuilder, Symbol},
  function, symbol,
  id::{Match, MatchSettings, Pattern, Replacement},
  utils::Settable,
};
```

Implementation rules:

- Use `Replacement::new(lhs.to_pattern(), rhs.to_pattern())` for local structural rules whose right side is another static pattern.
- Use `replace_multiple_into` in a loop for FORM-style short-circuit passes where rule order matters and each pass should be followed by expansion and metric simplification.
- Use `replace_map` or `ReplaceBuilder::with_map` for Chisholm parity, sequence reversal, trace recursion, `ReplaceLoop`-style loop detection, cyclic color traces, and fresh dummy allocation.
- Add `MatchSettings { rhs_cache_size: 0, ..Default::default() }` for replacement maps whose right side depends on match-local fresh symbols.
- Keep `chain` and `trace` non-symmetric. Their argument order is algebraically meaningful.

4.5.2 Gamma collection patterns

Mathematical identity: Equation 26. Source: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L604-L613.

Raw gamma pair:

```
gamma(bis(d_,a_),bis(d_,b_),x_)
* gamma(bis(d_,b_),bis(d_,c_),y_)
  -> chain(bis(d_,a_),bis(d_,c_),
       gamma(in,out,x_),
       gamma(in,out,y_))
```

Append and prepend raw gamma factors:

```
chain(bis(d_,a_),bis(d_,b_),xs___)
* gamma(bis(d_,b_),bis(d_,c_),y_)
  -> chain(bis(d_,a_),bis(d_,c_),xs___,gamma(in,out,y_))

gamma(bis(d_,a_),bis(d_,b_),x_)
* chain(bis(d_,b_),bis(d_,c_),ys___)
  -> chain(bis(d_,a_),bis(d_,c_),gamma(in,out,x_),ys___)
```

Join two chains:

```
chain(bis(d_,a_),bis(d_,b_),xs___)
* chain(bis(d_,b_),bis(d_,c_),ys___)
  -> chain(bis(d_,a_),bis(d_,c_),xs___,ys___)
```

Orientation case:

```
chain(bis(d_,a_),bis(d_,b_),xs___)
* chain(bis(d_,c_),bis(d_,b_),ys___)
  -> chain(bis(d_,a_),bis(d_,c_),xs___,reverse_flip(ys___))
```

`reverse_flip` is not a plain Symbolica replacement. Build it in Rust by reading the matched `ys___` argument list, reversing the entries, and swapping `in` and `out` in each tensor factor.

4.5.3 Gamma terminal and trace patterns

Mathematical identity: Equation 29.

```
chain(bis(d_,a_),bis(d_,a_),xs___)
  -> trace(bis(d_),xs___)

trace(bis(d_),gamma(in,out,x_))
  -> 0

trace(bis(4),
  gamma(in,out,mink(4,mu_)),
  gamma(in,out,mink(4,nu_)))
  -> 4 * g(mink(4,mu_),mink(4,nu_))
```

Adjacent repeated Lorentz object:

```
chain(bis(d_,a_),bis(d_,b_),
  xs___,
  gamma(in,out,mink(d_,mu_)),
```

```
  gamma(in,out,mink(d_,mu_)),
  ys___)
  -> d_ * chain(bis(d_,a_),bis(d_,b_),xs___,ys___)
```

GammaLoop/idenso currently has the equivalent repeated-object contraction in Rust at https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L784-L829 and trace closure at https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L877-L879. FORM's corresponding short-circuit in `Trace4Gen` removes adjacent equal objects before deeper recursion; source https://github.com/form-dev/form/blob/master/sources/opera.c#L958-L972.

4.5.4 Gamma Chisholm and trace recursion builders

Mathematical identities: Equation 33, Equation 34, and Equation 32. Source: https://github.com/form-dev/form/blob/master/sources/opera.c#L670-L680, https://github.com/form-dev/form/blob/master/sources/opera.c#L978-L1012, https://github.com/form-dev/form/blob/master/sources/opera.c#L1021-L1096, https://github.com/form-dev/form/blob/master/sources/opera.c#L1118-L1157.

Match contracted endpoints in four dimensions:

```
chain(bis(4,a_),bis(4,b_),
  gamma(in,out,mink(4,mu_)),
  middle___,
  gamma(in,out,mink(4,mu_)))
```

Rust-side builder:

```
fn build_chisholm(middle: &[Atom], a: Atom, b: Atom) -> Option<Atom> {
  if !all_gamma_factors(middle) {
    return None;
  }
  match middle.len() {
    n if n % 2 == 1 => {
      Some(-Atom::num(2) * chain(a, b, reverse(middle)))
    }
    2 => {
      let (x, y) = lorentz_pair(&middle[0], &middle[1])?;
      Some(Atom::num(4) * g(x, y) * bis_id(a, b))
    }
    n if n % 2 == 0 => {
      Some(Atom::num(2) * chain(a.clone(), b.clone(), move_last_to_front(middle))
        + Atom::num(2) * chain(a, b, reverse_except_last_then_last(middle)))
    }
    _ => None,
  }
}
```

Trace recursion should also be a Rust-side builder, not a static RHS:

```
trace(bis(4),first_,rest___)
```

Builder rule: if the trace length is odd, return zero. If the length is two, return `4 * g(first, second)`. Otherwise sum over all pairings of `first_` with one later gamma, alternating signs and recursively rebuilding `trace(bis(4), ...)`. This mirrors the source branch that uses zero/two-gamma terminals and then recursive generation; see https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432 and https://github.com/form-dev/form/blob/master/sources/opera.c#L830-L856.

4.5.5 Color collection patterns

Mathematical identity: Equation 37.

Raw generator pair:

```
t(coad(na_,a_),cof(nc_,i_),dind(cof(nc_,j_)))
* t(coad(na_,b_),cof(nc_,j_),dind(cof(nc_,k_)))
  -> chain(cof(nc_,i_),dind(cof(nc_,k_)),
       t(coad(na_,a_),in,out),
       t(coad(na_,b_),in,out))
```

Join chains:

```
chain(cof(nc_,i_),dind(cof(nc_,j_)),xs___)
* chain(cof(nc_,j_),dind(cof(nc_,k_)),ys___)
  -> chain(cof(nc_,i_),dind(cof(nc_,k_)),xs___,ys___)
```

Opposite endpoint orientation:

```
chain(cof(nc_,i_),dind(cof(nc_,j_)),xs___)
* chain(cof(nc_,k_),dind(cof(nc_,j_)),ys___)
  -> chain(cof(nc_,i_),dind(cof(nc_,k_)),xs___,reverse_flip(ys___))
```

The first two rules correspond to `color.h` line joining and trace closure at package lines 91–93. GammaLoop/idenso's current direct color simplifier starts from raw `spenso::t` and `spenso::f` patterns; source https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/color/mod.rs#L360-L414.

4.5.6 Color trace and Casimir patterns

Mathematical identity: Equation 38 and Equation 40.

```
chain(cof(nc_,i_),dind(cof(nc_,i_)),xs___)
  -> trace(cof(nc_),xs___)

trace(cof(nc_),t(coad(na_,a_),in,out))
  -> 0

trace(cof(nc_),
  t(coad(na_,a_),in,out),
  t(coad(na_,b_),in,out))
  -> TR * g(coad(na_,a_),coad(na_,b_))

trace(cof(nc_),
  t(coad(na_,a_),in,out),
  t(coad(na_,b_),in,out),
  t(coad(na_,c_),in,out))
  -> dR(coad(na_,a_),coad(na_,b_),coad(na_,c_))
     + i_ / 2 * TR * f(coad(na_,a_),coad(na_,b_),coad(na_,c_))
```

Casimir and separated-generator shortcuts:

```
chain(cof(nc_,i_),dind(cof(nc_,k_)),
  xs___,
  t(coad(na_,a_),in,out),
  t(coad(na_,a_),in,out),
  ys___)
```

```
  -> CF * chain(cof(nc_,i_),dind(cof(nc_,k_)),xs___,ys___)

trace(cof(nc_),
  t(coad(na_,a_),in,out),
  t(coad(na_,b_),in,out),
  t(coad(na_,a_),in,out),
  xs___)
  -> (CF - CA / 2) * trace(cof(nc_),t(coad(na_,b_),in,out),xs___)
```

Source: `color.h` selected-trace terminal rules at lines 459–464 and easy contraction rules at lines 108–111 and 173–175.

4.5.7 Color structure-constant and invariant patterns

Structure-constant contractions:

```
f(coad(na_,a_),coad(na_,c_),coad(na_,d_))
* f(coad(na_,b_),coad(na_,c_),coad(na_,d_))
  -> CA * g(coad(na_,a_),coad(na_,b_))

f(coad(na_,a_),coad(na_,b_),coad(na_,e_))
* f(coad(na_,b_),coad(na_,c_),coad(na_,f_))
* f(coad(na_,c_),coad(na_,a_),coad(na_,g_))
  -> CA / 2 * f(coad(na_,e_),coad(na_,f_),coad(na_,g_))
```

Trace–structure shortcut:

```
trace(cof(nc_),
  t(coad(na_,a_),in,out),
  t(coad(na_,b_),in,out),
  xs___)
* f(coad(na_,a_),coad(na_,b_),coad(na_,c_))
  -> i_ * CA / 2 * trace(cof(nc_),t(coad(na_,c_),in,out),xs___)
```

Symmetric-invariant contraction:

```
d(sym_x_,coad(na_,a_),coad(na_,b_),coad(na_,c_))
* d(sym_y_,coad(na_,a_),coad(na_,b_),coad(na_,d_))
  -> d33(sym_x_,sym_y_) * g(coad(na_,c_),coad(na_,d_)) / na_
```

`f` must be registered or canonicalized as antisymmetric. Symbolica's documentation warns that antisymmetric functions canonicalize written patterns; implementers should either use canonical argument order in the pattern or absorb the sign in a pre-canonicalization pass. Source for small `f` loop reductions: `color.h` lines 148–150, 208–210, 505–507, and 537–541. Source for `d33` and higher invariant contractions: `color.h` lines 1287–1348.

4.5.8 FORM short-circuit rewrite order

These are the high-priority short-circuit patterns FORM uses before falling back to expensive generic recursion.

Gamma `trace4` short-circuit order:

- Reject impossible input: negative number, odd number, or missing spin-line parameter returns zero. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L715-L717.
- Resolve zero- and two-gamma traces before recursion; gamma-five can force zero. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432 and https://github.com/form-dev/form/blob/master/sources/opera.c#L867-L875.
- Remove adjacent equal objects before Chisholm scans. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L958-L972.

- Try four-dimensional odd Chisholm contraction first. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L978-L1012.
- Try four-dimensional even Chisholm contraction, with a special two-interior case before the longer case. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L1021-L1096 and https://github.com/form-dev/form/blob/master/sources/opera.c#L1118-L1157.
- Search same objects by shortest cyclic distance and emit alternating recursive terms. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L1168-L1232.
- If all objects are different, fall back to `Trace4no`; the three-gamma epsilon trick is part of the four-dimensional branch. Source https://github.com/form-dev/form/blob/master/sources/opera.c#L320-L329 and https://github.com/form-dev/form/blob/master/sources/opera.c#L1240-L1244.

Color `color.h` short-circuit order:

- Join open color lines and close equal endpoints before any trace selection. Source `color.h` lines 91–93.
- Skip the main trace route unless at least one closed trace exists. Source `color.h` lines 95–103.
- Apply easy trace contractions first: adjacent equal generator, separated `a b a`, and trace times `f(a,b,c)`. Source `color.h` lines 108–111 and 173–175.
- Rewrite duplicated trace blocks of lengths four, three, and two before general expansion. Source `color.h` lines 112–147 and 178–207.
- Collapse small `f` loops via `ReplaceLoop`: two-leg and three-leg loops are terminal. Source `color.h` lines 148–150, 208–210, 505–507, 537–541, and 672–676.
- Select the trace with the fewest different indices by wrapping traces in `cOlTT`. Source `color.h` lines 156–163.
- Use terminal selected-trace rules for one, two, three, and four generators before converting longer traces back to `T`. Source `color.h` lines 459–466.
- In `simpli`, discard odd `f` counts, contract matching invariant-vector `f f` pairs, then run generalized Jacobi/invariant passes. Source `color.h` lines 688–724.
- Convert completed invariant-vector products into `d33`, `d44`, `d55`, and higher invariants, and discard impossible overlap topologies. Source `color.h` lines 1258–1294 and 1316–1348.

Rust implementation note: implement the short-circuit order as ordered passes, not as one unordered `replace_multiple` containing every rule. FORM's performance relies on reducing the expression before the broader searches are attempted.

4.6 FORM-side test cases in Symbolica/Spenso syntax

This section translates the compact FORM-side rule tests and the package example cases into the `chain`/`trace` notation used above. Expected values are written in the same symbolic convention: `TR` for `I2R`, `CA` for `cA`, `CF` for `cR`, and `NA` for adjoint dimension. The local validation used FORM 4.3.1 for the explicitly listed compact values.

4.6.1 Gamma trace tests

Source basis: FORM manual gamma algebra and `trace4` implementation; see https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012, https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432, and https://github.com/form-dev/form/blob/master/sources/opera.c#L867-L875.

```
// FORM: g_(1,mu,nu); trace4,1;
trace(bis(4),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)))
  -> 4 * g(mink(4,mu),mink(4,nu))

// FORM: g_(1,mu,nu,rho); trace4,1;
trace(bis(4),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)),
  gamma(in,out,mink(4,rho)))
```

```
  -> 0

// FORM: g_(1,mu,nu,rho,sigma); trace4,1;
trace(bis(4),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)),
  gamma(in,out,mink(4,rho)),
  gamma(in,out,mink(4,sigma)))
  -> 4 * g(mink(4,mu),mink(4,nu)) * g(mink(4,rho),mink(4,sigma))
   - 4 * g(mink(4,mu),mink(4,rho)) * g(mink(4,nu),mink(4,sigma))
   + 4 * g(mink(4,mu),mink(4,sigma)) * g(mink(4,nu),mink(4,rho))
```

Chisholm and adjacent-contraction chain tests:

```
// FORM manual: g_(1,mu,mu) = gi_(1)*d_(mu,mu).
chain(bis(d,a),bis(d,b),
  gamma(in,out,mink(d,mu)),
  gamma(in,out,mink(d,mu)))
  -> d * g(bis(d,a),bis(d,b))

// FORM manual, four-dimensional odd interior.
chain(bis(4,a),bis(4,b),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu1)),
  gamma(in,out,mink(4,nu2)),
  gamma(in,out,mink(4,nu3)),
  gamma(in,out,mink(4,mu)))
  -> -2 * chain(bis(4,a),bis(4,b),
       gamma(in,out,mink(4,nu3)),
       gamma(in,out,mink(4,nu2)),
       gamma(in,out,mink(4,nu1)))

// FORM manual, four-dimensional two-interior special case.
chain(bis(4,a),bis(4,b),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)),
  gamma(in,out,mink(4,rho)),
  gamma(in,out,mink(4,mu)))
  -> 4 * g(mink(4,nu),mink(4,rho)) * g(bis(4,a),bis(4,b))
```

Gamma-five epsilon test:

```
// FORM Trick identity in opera.c.
chain(bis(4,a),bis(4,b),
  gamma(in,out,mink(4,mu)),
  gamma(in,out,mink(4,nu)),
  gamma(in,out,mink(4,rho)))
  -> epsilon(mink(4,mu),mink(4,nu),mink(4,rho),mink(4,sigma))
     * chain(bis(4,a),bis(4,b),
         gamma5(in,out),
         gamma(in,out,mink(4,sigma)))
     + g(mink(4,mu),mink(4,nu))
       * chain(bis(4,a),bis(4,b),gamma(in,out,mink(4,rho)))
     - g(mink(4,mu),mink(4,rho))
       * chain(bis(4,a),bis(4,b),gamma(in,out,mink(4,nu)))
     + g(mink(4,nu),mink(4,rho))
       * chain(bis(4,a),bis(4,b),gamma(in,out,mink(4,mu)))
```

FORM manual gamma-five stress examples:

```
// FORM: symmetric trace of gamma5 and 12 regular matrices via distrib_ and tracen.
g5_trace_symmetric_12(m1,...,m12)
  -> 51975 terms

// FORM: regular trace g_(1,5_,m1,...,m12); trace4,1.
trace(bis(4),gamma5(in,out),gamma(in,out,mink(4,m1)),...,gamma(in,out,mink(4,m12)))
  -> 1029 output terms after FORM trace4
```

These are term-count regression tests from the manual, not compact algebraic values.

4.6.2 Compact color-rule tests

Source basis: `color.h` line joining, selected trace terminals, and small `f` loops; see https://www.nikhef.nl/~form/maindir/packages/color/color.h, especially lines 91–93, 459–464, and 505–507.

```
// FORM: T(i1,i1,a)
chain(cof(Nc,i),dind(cof(Nc,i)),
  t(coad(NA,a),in,out))
  -> 0

// FORM: T(i1,i1,a,b)
chain(cof(Nc,i),dind(cof(Nc,i)),
  t(coad(NA,a),in,out),
  t(coad(NA,b),in,out))
  -> TR * g(coad(NA,a),coad(NA,b))

// FORM: T(i1,i1,a,b,c)
chain(cof(Nc,i),dind(cof(Nc,i)),
  t(coad(NA,a),in,out),
  t(coad(NA,b),in,out),
  t(coad(NA,c),in,out))
  -> dR(coad(NA,a),coad(NA,b),coad(NA,c))
     + i_ / 2 * TR * f(coad(NA,a),coad(NA,b),coad(NA,c))
```

```
// FORM: f(a,c,d)*f(b,c,d)
f(coad(NA,a),coad(NA,c),coad(NA,d))
* f(coad(NA,b),coad(NA,c),coad(NA,d))
  -> CA * g(coad(NA,a),coad(NA,b))

// FORM: f(a,b,e)*f(b,c,f)*f(c,a,g)
f(coad(NA,a),coad(NA,b),coad(NA,e))
* f(coad(NA,b),coad(NA,c),coad(NA,f))
* f(coad(NA,c),coad(NA,a),coad(NA,g))
  -> CA / 2 * f(coad(NA,e),coad(NA,f),coad(NA,g))
```

Four-generator trace terminal:

```
trace(cof(Nc),
  t(coad(NA,a),in,out),
  t(coad(NA,b),in,out),
  t(coad(NA,c),in,out),
  t(coad(NA,d),in,out))
  -> dR(coad(NA,a),coad(NA,b),coad(NA,c),coad(NA,d))
     + i_ / 2 * (
         dR(coad(NA,a),coad(NA,b),coad(NA,x))
         * f(coad(NA,c),coad(NA,d),coad(NA,x))
         + dR(coad(NA,c),coad(NA,d),coad(NA,x))
         * f(coad(NA,a),coad(NA,b),coad(NA,x)))
```

```
    - TR / 6 * f(coad(NA,a),coad(NA,c),coad(NA,x))
      * f(coad(NA,b),coad(NA,d),coad(NA,x))
    + TR / 3 * f(coad(NA,a),coad(NA,d),coad(NA,x))
      * f(coad(NA,b),coad(NA,c),coad(NA,x))
```

4.6.3 FORM `color.tar.gz` example families

The color package page links `color.tar.gz` as "some FORM programs that use color.h"; see https://www.nikhef.nl/~form/maindir/packages/color/color.html and https://www.nikhef.nl/~form/maindir/packages/color/color.tar.gz. The example programs define source-side graph families. The following schematics preserve their topology in Symbolica/spenso notation.

`tloop.frm` cases:

```
// TYPE = qloop, SIZE = n.
chain(cof(Nc,i1),dind(cof(Nc,i1)),
  t(coad(NA,j1),in,out),...,t(coad(NA,jn),in,out),
  t(coad(NA,j1),in,out),...,t(coad(NA,jn),in,out))

// For SIZE = 3, FORM color+simpli expected:
Q6 -> NA * TR * CF^2 - 3/2 * NA * TR * CA * CF + 1/2 * NA * TR * CA^2

// TYPE = gloop, SIZE = n.
f(coad(NA,k1),coad(NA,k2),coad(NA,j1)) * ...
* f(coad(NA,k(2 n)),coad(NA,k1),coad(NA,jn))

// For SIZE = 3:
G6 -> 0

// TYPE = qqloop, SIZE = n: two fundamental loops linked by adjoint labels.
trace(cof(Nc),t(j1),...,t(jn)) * trace(cof(Nc),t(j1),...,t(jn))

// For SIZE = 3:
QQ3 -> -1/4 * NA * TR^2 * CA + d33(R1,R2)

// TYPE = qgloop, SIZE = n: one fundamental loop and one adjoint loop.
trace(cof(Nc),t(j1),...,t(jn)) * adjoint_loop(j1,...,jn)

// For SIZE = 3:
QG3 -> i_ / 4 * NA * TR * CA^2

// TYPE = ggloop, SIZE = n: two linked adjoint loops.
adjoint_loop(j1,...,jn) * adjoint_loop(j1,...,jn)

// For SIZE = 3:
GG3 -> 1/4 * NA * CA^3
```

Fixed graph examples:

```
// TYPE = g14: Coxeter graph with 14 trivalent adjoint vertices.
g14 -> 1/648 * NA * CA^7
    - 8/15 * d444(A1,A2,A3) * CA
    + 16/9 * d644(A1,A2,A3)

// TYPE = fiveq: five linked quark loops.
fiveq -> 1/192 * NA * TR^5 * CA^3
      + 1/4 * d33(R1,R2) * TR^3 * CA^2
      + 5/48 * d33(R1,R2) * d33(R3,R4) * NA^-1 * TR * CA
```

```
      + 5/48 * d33(R1,R3) * d33(R2,R4) * NA^-1 * TR * CA
      + 1/8 * d33(R1,R4) * d33(R2,R3) * NA^-1 * TR * CA
      + 1/16 * d44(R1,A1) * TR^4
      + 3/8 * d433(R3,R1,R2) * TR^2 * CA
      + 1/2 * d3333(R1,R2,R3,R4) * TR * CA
      + d43333a(R5,R2,R1,R4,R3)

// TYPE = g24 is a high-cost stress example in tloop.frm.
// No compact expected value is included in the package source; it is not a short unit
test.
```

`su.frm` examples using `#call SUn`:

```
// CASE = F3F3.
F3F3 -> 2*a^3*NF^-1 - 5/2*a^3*NF + 1/2*a^3*NF^3

// CASE = F4F4.
F4F4 -> -3*a^4*nf^2*NF^-2 + 4*a^4*nf^2
        - 7/6*a^4*nf^2*NF^2 + 1/6*a^4*nf^2*NF^4

// CASE = F4A4.
F4A4 -> -2*a^4*nf*NF + 5/3*a^4*nf*NF^3 + 1/3*a^4*nf*NF^5

// CASE = A4A4.
A4A4 -> -24*a^4*NF^2 + 70/3*a^4*NF^4 + 2/3*a^4*NF^6

// CASE = FnFn with n = 3.
F3F3 -> 2*a^3*nf^2*NF^-1 - 2*a^3*nf^2*NF

// CASE = FnAn with n = 3.
F3A3 -> -i_*a^3*nf*NF^2 + i_*a^3*nf*NF^4

// CASE = AnAn with n = 3.
A3A3 -> -2*a^3*NF^3 + 2*a^3*NF^5

// CASE = qloop with n = 3.
Q6 -> -a^3*nf*NF^-2 + a^3*nf*NF^2

// CASE = gloop with n = 3.
G6 -> 0
```

The SU checker procedure uses $T_R = a$, $C_F = a\frac{N_F^2-1}{N_F}$, $C_A = 2aN_F$, and $N_A = N_F^2 - 1$; source `SUn.prc` lines 26–58.

4.7 Implementation caveats

- FORM's four-dimensional gamma simplifications depend on index dimension checks in `Trace4Gen`; applying them in dimension-generic algebra is wrong.
- `trace4` accepts gamma-five branches with special sign changes and gamma-six/gamma-seven toggles; `tracen` is dimension-generic and does not tolerate gamma-five variants.
- `color.h` is group- and representation-invariant. It intentionally keeps symbols such as `cR`, `cA`, `I2R`, `NA`, and higher `d` invariants rather than reducing all cases to N_c.
- The `simpli` procedure is a rewrite strategy over invariant environments. Its output depends on canonicalization choices and on the `RANK` bound.
- Symbolica matching is syntactic. Sequence reversal, parity tests, fresh dummy generation, and antisymmetric signs require Rust-side construction or explicit pattern restrictions.
- The approved `chain` and `trace` syntax in this document is a target specification for tensor-pattern rules, not a claim that Symbolica ships built-in functions with those names.

4.8 Appendix: source map

Rule	Identity	Equation label	Primary source
G1	$\gamma(v)^{\beta}_{\alpha}\gamma(w)^{\chi}_{\beta} = (\Gamma^{vw})^{\chi}_{\alpha}$	Equation 25	https://github.com/form-dev/form/blob/master/sources/opera.c#L715-L745 https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L604-L613
G2	chain orientation	Equation 27	https://symbolica.io/docs/pattern_matching.html https://docs.rs/spenso/latest/spenso/
G3	$\gamma^{\mu}_{\alpha^{\beta}}\gamma^{\chi}_{\mu,\beta} = D\delta^{\chi}_{\alpha}$	Equation 28	https://github.com/form-dev/form/blob/master/sources/opera.c#L958-L972 https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L784-L829
G4	$(\Gamma^{v_1\cdots v_n})^{\alpha}_{\alpha}$	Equation 29	https://github.com/form-dev/form/blob/master/sources/opera.c#L730-L745 https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs#L877-L879
G5	odd trace zero, two trace	Equation 30, Equation 31	https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432 https://github.com/form-dev/form/blob/master/sources/opera.c#L830-L856
G6	Chisholm odd/even	Equation 33, Equation 34	https://github.com/form-dev/form/blob/master/sources/opera.c#L670-L680 https://github.com/form-dev/form/blob/master/sources/opera.c#L978-L1012 https://github.com/form-dev/form/blob/master/sources/opera.c#L1118-L1157

Rule	Identity	Equation label	Primary source
G7	gamma-five/equation epsilon trick	Equation 36	https://github.com/form-dev/form/blob/master/sources/opera.c#L320-L329 https://github.com/form-dev/form/blob/master/sources/opera.c#L480-L489
C1	open color-line joining	Equation 37	`color.h` lines 91–93, https://www.nikhef.nl/~form/maindir/packages/color/color.h https://www.nikhef.nl/~form/maindir/packages/color/color.html
C2	trace closure and T_R	Equation 38, Equation 40	`color.h` lines 459–461, https://www.nikhef.nl/~form/maindir/packages/color/color.h
C3	Casimir inside trace	Equation 41, Equation 42	`color.h` lines 108–111, https://www.nikhef.nl/~form/maindir/packages/color/color.h
C4	fundamental Fierz	Equation 43	https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/color/mod.rs#L407-L414
C5	ff and fff	Equation 44, Equation 45	`color.h` lines 148–150 and 505–507, https://www.nikhef.nl/~form/maindir/packages/color/color.h
C6	trace–f contraction	Equation 46	`color.h` lines 108–111 and 173–175, https://www.nikhef.nl/~form/maindir/packages/color/color.h
C7	`d33` contractions	Equation 47, Equation 48	`color.h` lines 1287–1294 and 1316–1335, https://www.nikhef.nl/~form/maindir/packages/color/color.h
C8	`simpli` strategy	`<rule-color-simpli>`	`color.h` lines 688–713, https://www.nikhef.nl/~form/maindir/packages/color/color.h
S1	Rust Symbolica pattern API	Symbolica + spenso section	https://docs.rs/symbolica/latest/symbolica/id/index.html https://docs.rs/symbolica/latest/symbolica/id/struct.ReplaceBuilder.html

Rule	Identity	Equation label	Primary source
S2	spenso symbolic tensor structures	Symbolica + spenso section	https://docs.rs/spenso/latest/spenso/ https://docs.rs/spenso/latest/src/spenso/network/library/symbolic.rs.html
S3	FORM gamma short-circuit order	FORM short-circuit section	https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432 https://github.com/form-dev/form/blob/master/sources/opera.c#L958-L972 https://github.com/form-dev/form/blob/master/sources/opera.c#L1168-L1232
S4	FORM color short-circuit order	FORM short-circuit section	`color.h` lines 91–163, 459–507, 688–724, and 1258–1348, https://www.nikhef.nl/~form/maindir/packages/color/color.h
T1	FORM gamma test values	FORM-side test section	https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012 https://github.com/form-dev/form/blob/master/sources/opera.c#L424-L432
T2	FORM compact color test values	FORM-side test section	`color.h` lines 459–464 and 505–507, https://www.nikhef.nl/~form/maindir/packages/color/color.h
T3	FORM color example families	FORM-side test section	`color.tar.gz`: `tloop.frm`, `su.frm`, `SUn.prc`, https://www.nikhef.nl/~form/maindir/packages/color/color.tar.gz

Source snippets appear next to each detailed rule above. Symbolica/spenso pattern references are the code blocks in the corresponding detailed rule and in the Symbolica + spenso rewrite-pattern section.

4.9 Appendix: validation

4.9.1 Typst compile result

The pre-migration standalone source recorded this successful environment:

```
typst 0.14.2 (b33de9de)
```

Its captured compilation command was:

```
typst compile form_gamma_color_rules_from_scratch.typ
form_gamma_color_rules_from_scratch.pdf
```

That historical command succeeded and produced `form_gamma_color_rules_from_scratch.pdf`; the old filename is evidence, not the current source path. The maintained specification now builds through the Idenso product route and complete PDF with:

```
cargo run --locked -p alphal00p-docs-builder -- build --product idenso --channel latest --
output /tmp/idenso-docs --skip-rustdoc
```

4.9.2 Static checks

Checks captured against the pre-migration Typst filename:

```
rg -n '\b\x70rod\b' form_gamma_color_rules_from_scratch.typ
rg -n 'tr_[A-Za-z]+\(|Tr_[A-Za-z]+\(|op\("tr"\)_[A-Za-z]+\('
form_gamma_color_rules_from_scratch.typ
rg -n '#L[0-9]+' form_gamma_color_rules_from_scratch.typ
rg -n 'gamma_chain|gamma_trace' form_gamma_color_rules_from_scratch.typ
rg -n 'with_map|replace_map|FORM Short-Circuit' form_gamma_color_rules_from_scratch.typ
rg -n 'FORM-Side Test Cases|Q6 ->|F3F3 ->|g14 ->|fiveq ->'
form_gamma_color_rules_from_scratch.typ
```

Results:

- No word token `\x70rod` was found; intended occurrences use `product`.
- No subscripted trace operator was immediately attached to a parenthesized argument; trace calls use explicit spacing.
- GitHub line-anchor fragments were found for the FORM `opera.c`, GammaLoop Dirac, and GammaLoop color source links. `color.h` rules cite exact package-source line numbers because no official GitHub-hosted `color.h` was found in `form-dev/form`.
- The internal names `spenso::gamma_chain` and `spenso::gamma_trace` occur only in source-code snippets or explanatory text documenting GammaLoop internals; target patterns use `chain` and `trace`.
- Rust-only operations that cannot be static Symbolica RHS patterns, including sequence reversal, parity checks, cyclic loop detection, and fresh-index creation, are assigned to `with_map`, `replace_map`, or explicit Rust builders.
- The FORM-side test-case section is present and contains expected values for compact gamma tests, compact color tests, and short `color.tar.gz` example instantiations.

Additional FORM checks used to populate the test-case section:

```
form -q /tmp/gamma-test.frm
form -q /tmp/color-trace.frm
form -q /tmp/color-short.frm
cd /tmp/form-color && form -q tloop-qloop.frm
cd /tmp/form-color && form -q tloop-gloop.frm
cd /tmp/form-color && form -q tloop-qqloop.frm
cd /tmp/form-color && form -q tloop-qgloop.frm
cd /tmp/form-color && form -q tloop-ggloop.frm
cd /tmp/form-color && form -q tloop-g14.frm
cd /tmp/form-color && form -q tloop-fiveq.frm
cd /tmp/form-color && form -q su-F3F3.frm
cd /tmp/form-color && form -q su-F4F4.frm
cd /tmp/form-color && form -q su-F4A4.frm
cd /tmp/form-color && form -q su-A4A4.frm
cd /tmp/form-color && form -q su-FnFn.frm
cd /tmp/form-color && form -q su-FnAn.frm
cd /tmp/form-color && form -q su-AnAn.frm
```

```
cd /tmp/form-color && form -q su-qloop.frm
cd /tmp/form-color && form -q su-gloop.frm
```

4.9.3 Unresolved uncertainties

- The current official FORM GitHub repository contains `sources/opera.c` but not the legacy `color.h` package file. The package page links the actual source. Therefore color rules cite exact package-source line numbers and the package source URL, but they do not have official GitHub line anchors.
- The approved `chain` and `trace` notation is a specification-level Symbolica/spenso tensor-pattern notation. Exact Rust constructors for sequence reversal and parity-filtered Chisholm replacement must be implemented with Rust-side match processing.

4.10 References

- FORM manual gamma algebra: https://www.nikhef.nl/~form/maindir/documentation/reference/html/manual.html#dx1-85012.
- FORM `opera.c`: https://github.com/form-dev/form/blob/master/sources/opera.c.
- FORM `color.h` package page: https://www.nikhef.nl/~form/maindir/packages/color/color.html.
- FORM `color.h` source: https://www.nikhef.nl/~form/maindir/packages/color/color.h.
- FORM `color.h` example programs: https://www.nikhef.nl/~form/maindir/packages/color/color.tar.gz.
- GammaLoop/idenso Dirac implementation: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/dirac/mod.rs.
- GammaLoop/idenso color implementation: https://github.com/alphal00p/gammaloop/blob/cbfc3a5827679de14e9373edd6b332e6b9b28a55/crates/idenso/src/color/mod.rs.
- Symbolica pattern matching: https://symbolica.io/docs/pattern_matching.html.
- Symbolica Rust `id` API: https://docs.rs/symbolica/latest/symbolica/id/index.html.
- Symbolica Rust `ReplaceBuilder`: https://docs.rs/symbolica/latest/symbolica/id/struct.ReplaceBuilder.html.
- spenso crate documentation: https://docs.rs/spenso/latest/spenso/.
- spenso symbolic tensor-library source: https://docs.rs/spenso/latest/src/spenso/network/library/symbolic.rs.html.
- Typst syntax: https://typst.app/docs/reference/syntax/.
- Typst math: https://typst.app/docs/reference/math/.

5 Rust and Python APIs

5.1 Rust package

The `idenso` crate exposes representation types and syntax macros together with several rewrite families:

- `IndexTooling` covers canonicalization, conjugation, index wrapping, and dangling-index inspection for Symbolica atoms;
- `Cookable`, `CookSettings`, and the cook filters control reversible or flattening encodings;
- `SelectiveExpand` expands terms by recognized representation families;
- `dirac`, `color`, `epsilon`, and shorthand modules implement algebra-specific rewrites;
- `representations::initialize` installs the standard representation and tensor symbols.

Representation helper macros such as `bis!`, `cof!`, and `coad!` construct the symbolic forms expected by Spenso and Idenso. The curated Rust API gives their accepted forms, return types, feature gates, and source locations. APIs behind `bincode`, `reference-cases`, `python`, and `python_stubgen` are available only when the matching Cargo feature is enabled.

5.2 Python community module

Part of Symbolica community
Import Idenso from `symbolica.community.idenso`. The `idenso` Cargo package supplies this community module when built with its Python feature; it is not a standalone PyPI distribution and is not included in the GammaLoop wheel merely because the crates share a repository.

Install a Symbolica Python distribution only if its release manifest says it includes Idenso, then verify the actual assembly with `python -c "import symbolica.community.idenso"`. There is no `pip install idenso` fallback. For a source build, add this crate to the external symbolica-community assembly with the `python` feature and call `IdensoModule`'s `SymbolicaCommunityModule` registration when that extension is assembled. Building the Rust crate alone does not add the community module to an already installed Symbolica package.

The generated Python API records exact signatures and defaults. Its operations group naturally into four phases:

- setup: `initialize`;
- expansion: `expand_bis`, `expand_mink`, `expand_mink_bis`, `expand_metrics`, and `expand_color`;
- index preparation: `wrap_indices`, `wrap_dummies`, `list_dangling`, `cook_indices`, and `cook_function`;
- algebra: `dirac_adjoint`, `simplify_gamma`, `to_dots`, `simplify_metrics`, and `simplify_color`.

```python
from symbolica.community.idenso import (
    initialize,
    list_dangling,
    simplify_metrics,
)

initialize()
# `expr` is a Symbolica Expression written with Spenso-compatible tensor forms.
external_indices = list_dangling(expr)
reduced = simplify_metrics(expr)
```

The example deliberately leaves expression construction to Symbolica and Spenso: Idenso does not define a second parser syntax. Apply one transformation at a time while developing a pipeline so expression growth and convention changes remain observable.

For implementation details, start with the Rust API and the Python binding.

6 Version history

History coverage
This version of Idenso is 0.3.0, but the published changelog currently ends at 0.2.5. Release notes for 0.3.0 are not yet available, so the history below is incomplete for that version.

6.1 Available history

The rendered Idenso history records the published history through 0.2.5 and links to its canonical source.

When upgrading Idenso, pay particular attention to representation initialization, dummy-index handling, canonicalization, and Dirac, metric, and color conventions. A rewrite change can alter the shape or ordering of a symbolic result without changing the mathematical intent, so callers that persist or compare printed expressions should verify their chosen canonical form.

6.2 Upgrade checklist

Re-run representative identities and verify initialization, dummy-index allocation, canonicalization, and printed-expression expectations.

6.3 Reproducibility record

For reproducible upgrades, record the Idenso version and enabled features with the calculation. Do not assume that changes between 0.2.5 and 0.3.0 are fully represented in the available notes.

7 Idenso versions

This page renders the canonical Idenso changelog source as part of the Idenso website, navigation, and search.

All notable changes to this project will be documented in this file.

The format is based on Keep a Changelog, and this project adheres to Semantic Versioning.

8.1 Unreleased

8.2 0.2.5 - 2026-01-08

8.2.1 Fixed

- fix metric initialization and update to symbolica 1.2
- fix initialize to also load ETS

8.2.2 Other

- update linnet

8.3 0.2.4 - 2025-12-15

8.3.1 Fixed

- fix canonization in presence of duals
- fix gamma normalisation
- fix warnings
- fix `add_assign` with sparse tensors
- fix projp

8.3.2 Other

- Import initialize function
- Update symbolica to 1.1.0
- update linnet
- align to atom in argument of cof for Nc
- update linnet
- add readmes
- formatting and add ref for parametric
- update to 0.17 `pyo3_stub_gen`
- update linnet and make classattr to classmethods
- proper imports and feature flags for python
- move initialize to api
- Release spenso-macros 0.3
- update to symbolica 1.0
- update sympolica to 0.20
- Properly precontract scalars
- Add back pyo3-stub-gen-derive
- update to current dev symbolica and add pattern for pslash + m conjugate
- update to linnet 0.13.1

- working canonize (up to functions) and dirac adjoint
- spenso canonize buggy
- `robust_conj` but with lots of `gamma_0s`
- first try conj
- working pow execution with wrapping
- add function generic key

8.4 0.2.2 - 2025-08-18

8.4.1 Fixed

- fix cook indices
- fix clippy errors

8.4.2 Other

- update linnet to crates
- all tests pass
- Fix Multiple Heads Bug
- update linnet
- update linnet
- update linnet
- update parse problem and add spenso:: to all symbols
- make all symbols take spenso namespace
- operate directly on coefficient list
- remove expand

8.5 0.2.1 - 2025-07-15

8.5.1 Fixed

- fix gamma algebra again

8.5.2 Other

- update versions

8.6 0.2.0 - 2025-07-15

8.6.1 Fixed

- fix `wrap_indices`
- fix gamma algebra≈

8.6.2 Other

- allow arbitrary arguments for to dots

API reference

The packages available in this version are listed below. Follow an item link for its complete language reference.

`idenso`

Idenso's supported representation and algebra API. The manual owns the representation conventions; this scope provides *stable* source links.

`Index tooling`

```
Adds canonicalization, wrapping, conjugation, and dangling-index inspection to
Symbolica atoms.
```

```
Defines operations related to manipulating abstract indices within symbolic
expressions,
particularly relevant for physics calculations involving tensor structures and
diagrams.

This trait provides methods for conjugating expressions, wrapping indices, and
identifying
external ("dangling") indices.
```

```
pub trait IndexTooling {}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_index_tools<T: idenso::IndexTooling>(_expression: &T) {}
```

`canonize`

Kind: `Method`

```
fn canonize < Aind : AbsInd + ParseableAind + DummyAind >
(& self, new_dummy : impl FnMut(usize) -> Aind,) -> Atom;
```

`wrap_indices`

Kind: `Method`

```
fn wrap_indices(& self, header : Symbol) -> Atom;
```

```
Wraps all abstract indices within the expression using a specified header symbol.

This transforms indices like `mink(dim,idx)` into `mink(dim,header(idx))`. Useful for
distinguishing
between different copies of an expression, e.g., an amplitude and its complex
conjugate.

# Arguments
* `header` - The [`Symbol`] to use as the wrapping function name.

# Returns
A new [`Atom`] with all indices wrapped.
```

`spenso_conj`

Kind: `Method`

```
fn spenso_conj(& self) -> Atom;
```

spenso_print

Kind: Method

```
fn spenso_print < 'a > (& 'a self, settings : & SpensoPrintSettings) ->
AtomPrinter < 'a >;
```

wrap_dummies

Kind: Method

```
fn wrap_dummies < Aind : AbsInd + DummyAind + ParseableAind >
(& self, header : Symbol) -> Atom;
```

Wraps only the dummy (contracted) abstract indices within the expression using a header symbol.

Identifies indices that appear contracted (e.g., one covariant, one contravariant) and wraps only those, leaving external indices unchanged. Transforms `idx -> header(idx)`
for dummy indices `idx`.

Arguments
* `header` - The [`Symbol`] to use as the wrapping function name for dummy indices.

Returns
A new [`Atom`] with only dummy indices wrapped.

dirac_adjoint

Kind: Method

```
fn dirac_adjoint < Aind : DummyAind + ParseableAind + AbsInd > (& self) ->
eyre :: Result < Atom >;
```

Computes the physics-aware conjugate of the expression.

Applies conjugation rules specific to physics objects like spinors, gamma matrices, color representations, and the imaginary unit `i`. See implementation details for specific rules applied.

Returns
A new [`Atom`] representing the conjugated expression.

conjugate_transpose

Kind: Method

```
fn conjugate_transpose(& self, rep : impl RepName) -> Atom;
```

list_dangling

Kind: Method

```
fn list_dangling < Aind : AbsInd + DummyAind + ParseableAind > (& self) -> Vec
< Atom >;
```

Identifies and returns a list of dangling (external, uncontracted) indices.

Analyzes the expression to find indices that are not summed over. Returns them as `Atom`s. Note that dual indices might be represented wrapped in a `dind` function.

Returns
A `Vec<Atom>` where each `Atom` represents a dangling index.

alias_subtensors

Kind: Method

```
fn alias_subtensors(& self, tensor_name : & str) -> AliasedAtom;
```

Exhaustive reference · Source

Index cooking settings

```
Controls reversible, flattened, and filtered encodings of tensor indices.

Settings for cooking functions and representation-slot indices.

Tags are attached through `SymbolBuilder::new(...).with_tags(...)`, so they must be
fully namespaced Symbolica tags such as `idenso::cooked`.

pub struct CookSettings {}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let settings = idenso::CookSettings::reversible();
```

Exhaustive reference · Source

Cookable expressions

```
Applies or reverses index-cooking transformations on symbolic expressions.

Convenience methods for cooking Symbolica atoms.

pub trait Cookable {}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_cookable<T: idenso::Cookable>(_expression: &T) {}
```

cook

Kind: Method

```
fn cook(& self) -> Atom;

Reversibly cook function-like subexpressions with the default cooked tag.
```

cook_with_settings

Kind: Method

```
fn cook_with_settings(& self, settings : & CookSettings) -> Atom;

Cook function-like subexpressions with explicit settings.
```

uncook

Kind: Method

```
fn uncook(& self) -> Atom;

Undo default reversible cooking.
```

uncook_with_settings

Kind: Method

```
fn uncook_with_settings(& self, settings : & CookSettings) -> Atom;
```

Undo reversible cooking that used explicit settings.

cook_indices

Kind: Method

```
fn cook_indices(& self) -> Atom;
```

Flatten cook abstract-index payloads in recognized representation slots.

cook_indices_with_settings

Kind: Method

```
fn cook_indices_with_settings(& self, settings : & CookSettings) -> Atom;
```

Cook representation-slot index payloads with explicit settings.

cook_function

Kind: Method

```
fn cook_function(& self) -> Result < Atom, CookingError >;
```

Flatten cook a single atom into one symbol.

cook_function_with_settings

Kind: Method

```
fn cook_function_with_settings(& self, settings : & CookSettings) -> Result <
Atom, CookingError >;
```

Cook a single atom into one symbol with explicit settings.

Exhaustive reference · Source

Selective expansion

Expands only tensor families selected by their representation.

Expands only tensor families selected by their representation.

```
pub trait SelectiveExpand {}
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
fn accepts_expansion<T: idenso::selective_expand::SelectiveExpand>(_expression: &T)
{}
```

expand_in_patterns

Kind: Method

```
fn expand_in_patterns(& self, pats : & [Pattern]) -> Vec < (Atom, Atom) >;
```

expand_metrics

Kind: Method

```
fn expand_metrics(& self) -> Vec < (Atom, Atom) > { … }
```

expand_bis

Kind: Method

```
fn expand_bis(& self) -> Vec < (Atom, Atom) > { … }
```

expand_mink

Kind: Method

```
fn expand_mink(& self) -> Vec < (Atom, Atom) > { … }
```

expand_mink_bis

Kind: Method

```
fn expand_mink_bis(& self) -> Vec < (Atom, Atom) > { … }
```

Exhaustive reference · Source

Bispinor representation macro

Builds stripped or indexed bispinor representation atoms in Spenso syntax.

Builds a stripped or indexed bispinor representation atom.

```
macro_rules! bis { … }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::bis!(4);
```

Exhaustive reference · Source

Color-fundamental macro

Builds stripped or indexed color-fundamental representation atoms.

Builds a stripped or indexed color-fundamental representation atom.

```
macro_rules! cof { … }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::cof!(Nc);
```

Exhaustive reference · Source

Color-adjoint representation macro

Builds stripped or indexed color-adjoint representation atoms.

Builds a stripped or indexed color-adjoint representation atom.

```
macro_rules! coad { … }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let representation = idenso::coad!(Na);
```

Exhaustive reference · Source

Dirac gamma macro

Builds a Dirac gamma chain factor or an explicitly indexed gamma tensor.

Builds a gamma matrix.

```
With one argument, this builds a chain factor using the placeholder indices
`in` and `out`; use this form only as a factor inside `chain!` or `trace!`.
With three arguments, this builds the ordinary gamma tensor with explicit
spinor endpoints and a Lorentz slot.

Arguments are converted through `spenso::shadowing::IntoAtom`, so they
can be typed slots, atoms, or atom views.

# Examples

```ignore
use idenso::gamma;
use spenso::{chain, slot};

let factor = gamma!(slot!(mink4, mu));
let chain_expr = chain!(slot!(bis4, a), slot!(bis4, b), factor);
let default_tensor = gamma!(mu, a, b);
let indexed_tensor = gamma!(1, 2, 3);
let pattern_tensor = gamma!(RS.a__, RS.b__, RS.c__);
let pattern_tensor_with_slots = gamma!(RS.a__, [RS.d_, RS.i_], [RS.d_, RS.j_]);
let mixed_tensor = gamma!(mu, slot!(bis_d, a), 1);
let explicit_tensor = gamma!(slot!(mink_d, mu), slot!(bis_d, a), slot!(bis_d, b));
```

macro_rules! gamma { … }
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::gamma;
use spenso::{chain, slot};

let factor = gamma!(slot!(mink4, mu));
let chain_expr = chain!(slot!(bis4, a), slot!(bis4, b), factor);
let default_tensor = gamma!(mu, a, b);
let indexed_tensor = gamma!(1, 2, 3);
let pattern_tensor = gamma!(RS.a__, RS.b__, RS.c__);
let pattern_tensor_with_slots = gamma!(RS.a__, [RS.d_, RS.i_], [RS.d_, RS.j_]);
let mixed_tensor = gamma!(mu, slot!(bis_d, a), 1);
let explicit_tensor = gamma!(slot!(mink_d, mu), slot!(bis_d, a), slot!(bis_d, b));
```

Supported usage

```
let tensor = idenso::gamma!(1, 2, 3);
```

Exhaustive reference · Source

`Gamma-zero macro`

```
Builds a gamma-zero chain factor or an explicitly indexed gamma-zero tensor.

Builds a gamma-zero factor or tensor.

With no arguments, this builds a chain factor using the placeholder indices
`in` and `out`; use this form only as a factor inside `chain!` or `trace!`.
With two arguments, this builds the ordinary gamma-zero tensor between
explicit spinor endpoints.

Endpoints can be typed slots, atoms, atom views, default four-dimensional
```

```
bispinor identifiers/literals, pattern variables such as `RS.i__`, or
bracketed bispinor argument lists such as `[RS.d_, RS.i_]`.

# Examples

```ignore
use idenso::gamma0;

let factor = gamma0!();
let default_tensor = gamma0!(i, j);
let pattern_tensor = gamma0!(RS.i__, RS.j__);
let dimensioned_pattern_tensor = gamma0!([RS.d_, RS.i_], [RS.d_, RS.j_]);
```

macro_rules! gamma0 { … }
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::gamma0;

let factor = gamma0!();
let default_tensor = gamma0!(i, j);
let pattern_tensor = gamma0!(RS.i__, RS.j__);
let dimensioned_pattern_tensor = gamma0!([RS.d_, RS.i_], [RS.d_, RS.j_]);
```

Supported usage

```
let tensor = idenso::gamma0!(1, 2);
```

Exhaustive reference · Source

`Gamma-five macro`

```
Builds a gamma-five chain factor or an explicitly indexed gamma-five tensor.

Builds a gamma-five factor or tensor.

With no arguments, this builds a chain factor using the placeholder indices
`in` and `out`; the surrounding chain or trace owns the physical endpoints.
With two arguments, this builds the ordinary gamma-five tensor between
explicit spinor endpoints.

Pattern variables such as `RS.i__` are wrapped as bispinor arguments.

macro_rules! gamma5 { … }
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Supported usage

```
let tensor = idenso::gamma5!(1, 2);
```

Exhaustive reference · Source

`Levi-Civita tensor macro`

```
Builds an arbitrary-rank antisymmetric epsilon tensor from atom-like arguments.

Builds an antisymmetric epsilon tensor with arbitrary rank.
```

```
Arguments are converted through `spenso::shadowing::IntoAtom`, so tests
and rewrite code can pass slots, atoms, atom views, symbols, and integers
without spelling out the conversion.

macro_rules! epsilon { … }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
let tensor = idenso::epsilon!(1, 2, 3, 4);
```

Exhaustive reference · Source

`Color-generator macro`

```
Builds a color-generator chain factor or an explicitly indexed generator tensor.

Builds a color-generator atom.

With one argument, this builds a fundamental generator factor for use inside
`chain!` or `trace!`. The factor uses the chain placeholder indices `in` and
`out`; the surrounding chain or trace owns the physical endpoints.

With three arguments, this builds an explicit generator `t(a,i,j)`. The first
argument is an adjoint color index, the second is a fundamental color index,
and the third is an anti-fundamental color index.

Arguments can be typed slots, atoms, atom views, pattern variables such as
`RS.a__`, or bracketed representation argument lists such as
`[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so
local slot bindings and numeric labels are used as-is.

# Examples

```ignore
use idenso::color_t;
use spenso::{slot, trace};

let factor = color_t!(slot!(coad_na, a));
let expr = trace!(&cof_nc, factor);
let factor_from_binding = color_t!(slot!(coad_na, a));
let explicit_tensor = color_t!(slot!(coad_na, a), slot!(cof_nc, i), slot!
(cof_nc.dual(), j));
let pattern_tensor = color_t!(RS.a__, RS.i__, RS.j__);
let dimensioned_pattern = color_t!([CS.adj_, RS.a_], [CS.nc_, RS.i_], [CS.nc_,
RS.j_]);
```

macro_rules! color_t { … }
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::color_t;
use spenso::{slot, trace};

let factor = color_t!(slot!(coad_na, a));
```

```
let expr = trace!(&cof_nc, factor);
let factor_from_binding = color_t!(slot!(coad_na, a));
let explicit_tensor = color_t!(slot!(coad_na, a), slot!(cof_nc, i), slot!
(cof_nc.dual(), j));
let pattern_tensor = color_t!(RS.a__, RS.i__, RS.j__);
let dimensioned_pattern = color_t!([CS.adj_, RS.a_], [CS.nc_, RS.i_], [CS.nc_,
RS.j_]);
```

Supported usage

```
let tensor = idenso::color_t!(1, 2, 3);
```

Exhaustive reference · Source

`Color structure-constant macro`

```
Builds an explicitly indexed adjoint color structure-constant tensor.

Builds an adjoint color structure-constant atom `f(a,b,c)`.

Arguments can be typed slots, atoms, atom views, pattern variables such as
`RS.a__`, or bracketed adjoint representation argument lists such as
`[CS.adj_, RS.a_]`. Bare Rust identifiers and literals are expressions, so
local slot bindings and numeric labels are used as-is.

# Examples

```ignore
use idenso::color_f;
use spenso::slot;

let expr = color_f!(slot!(coad_na, a), slot!(coad_na, b), slot!(coad_na, c));
let numeric_label_tensor = color_f!(1, 2, 3);
let pattern_tensor = color_f!(RS.a__, RS.b__, RS.c__);
let dimensioned_pattern = color_f!([CS.adj_, RS.a_], [CS.adj_, RS.b_], [CS.adj_,
RS.c_]);
```

macro_rules! color_f { … }
```

Reference feature matrix: `bincode`, `reference-cases` (item is available without an additional gate)

Example 1

```
use idenso::color_f;
use spenso::slot;

let expr = color_f!(slot!(coad_na, a), slot!(coad_na, b), slot!(coad_na, c));
let numeric_label_tensor = color_f!(1, 2, 3);
let pattern_tensor = color_f!(RS.a__, RS.b__, RS.c__);
let dimensioned_pattern = color_f!([CS.adj_, RS.a_], [CS.adj_, RS.b_], [CS.adj_,
RS.c_]);
```

Supported usage

```
let tensor = idenso::color_f!(1, 2, 3);
```

Exhaustive reference · Source

`Initialize representations`

```
Initializes Idenso's standard representations and tensor symbols.
```

```
Register Idenso's built-in representations and algebra symbols with Symbolica.

Symbolica calls this during community-module initialization. Calling it again
is safe and ensures the standard Lorentz, spinor, and color objects exist.

fn initialize()
```

Reference feature matrix: `bincode, reference-cases` (item is available without an additional gate)

Supported usage

```
idenso::representations::initialize();
```

Exhaustive reference · Source

`idenso-community`

```
Exports registered by idenso.
```

`cook_function`

```
Convert a single function call into a flattened variable symbol.

Convert a single function call into a flattened variable symbol.

Transforms a function expression with arguments into a single symbolic variable
whose name encodes both the function name and its arguments. This is the
atomic version of `cook_indices()`, operating on individual function calls
rather than complete expressions.

**Function Cooking Transform:**
- Simple function: `f(a, b)` → `f_a_b`
- Nested arguments: `tensor(rep(mu))` → `tensor_rep_mu`
- Multiple arguments: `gamma(mu, alpha, beta)` → `gamma_mu_alpha_beta`
- Complex names: `my_function(x, y)` → `my_function_x_y`


**Constraints:**
- Input must be a single function call (not sum, product, etc.)
- Arguments must be cookable (symbols, numbers, simple functions)
- Cannot cook expressions containing polynomials or complex structures

# Arguments
- `self_`: expression representing a single function call to cook

# Returns
Expression containing the flattened variable symbol.

# Raises
`TypeError` if input is not a cookable function or contains invalid argument types.

# Examples:
```python
import symbolica as sp
from symbolica.community.idenso import cook_function

# Simple function cooking
f = sp.S('f')
a, b = sp.S('a','b')
```
```

```
cooked = cook_function(f(a, b))
print(cooked)  # Outputs: f_a_b
```

```
def cook_function(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`cook_indices`

`Convert complex nested index structures into flattened symbolic names.`

`Convert complex nested index structures into flattened symbolic names.`

```
Transforms hierarchical index expressions within tensor function arguments
into simplified, flat symbolic representations. This "cooking" process is
essential for pattern matching, simplification, and computational efficiency
when dealing with complex tensor expressions.

**Index Cooking Transformation:**
- Nested structure: `mink(4, f(g(h(μ))))` → `mink(4, f_g_h_mu)`
- Function chains: `lorentz(up(mu))` → `lorentz(up_mu)`
- Complex arguments: `tensor(rep(dim,type(idx)))` → `tensor(rep(dim,type_idx))`

**Scope:**
- Only affects indices appearing as function arguments
- Preserves top-level function structure
# Arguments
- `self_`: expression containing complex nested index structures

# Returns
Expression with flattened, simplified index names.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import wrap_indices, cook_indices

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, x)  # T(mu, nu; x)
print(tensor_with_args)
print(
    cook_indices(wrap_indices(tensor_with_args.to_expression(), sp.S("wrap")))
)
```
```

```
def cook_indices(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

dirac_adjoint

Return the Dirac adjoint of a Symbolica tensor expression.

Return the Dirac adjoint of a Symbolica tensor expression.

This reverses the spinor chain and applies the representation-aware adjoint
rules used by Idenso's Dirac algebra.

```
def dirac_adjoint(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

expand_bis

Expands factorized terms containing Dirac bispinor indices.

Expands factorized terms containing Dirac bispinor indices.

Finds and expands factorized expressions involving bispinor tensors and spinors,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

```
**Factorization Expansion:**
- `ψ(α) * (γ(μ,α,β) + σ(μ,α,β)) → ψ(α)*γ(μ,α,β) + ψ(α)*σ(μ,α,β)`
- Nested products with bispinor indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for gamma matrix simplification



# Arguments
- `self_`: expression containing factorized terms with bispinor indices

# Returns
Expanded expression with factorizations unfolded.
```

```
def expand_bis(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

expand_color

Expands factorized terms containing SU(N) color indices.

Expands factorized terms containing SU(N) color indices.

Finds and expands factorized expressions involving color tensors and fields,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

```
**Factorization Expansion:**
- `q(a) * (T(b,a,c) + S(b,a,c)) → q(a)*T(b,a,c) + q(a)*S(b,a,c)`
- Nested products with color indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for color algebra simplification

**Applications:**
- Expanding factorized QCD expressions before simplification
- Preparing for SU(N) algebra algorithms
- Unfolding nested products in gauge theory calculations
```

```
- Algebraic manipulation of color structures

# Arguments
- `self_`: expression containing factorized terms with color indices

# Returns
Expanded expression with factorizations unfolded.
```

```
def expand_color(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

expand_metrics

```
Expands factorized terms containing metric tensors.
```

```
Expands factorized terms containing metric tensors.

Finds and expands factorized expressions involving metric tensors and related
geometric objects, unfolding multiplicative structures for subsequent simplification.

# Arguments
- `self_`: expression containing factorized metric terms

# Returns
The expanded expression with metric factorizations unfolded.
```

```
def expand_metrics(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

expand_mink

```
Expands factorized terms containing Minkowski spacetime indices.
```

```
Expands factorized terms containing Minkowski spacetime indices.

Finds and expands factorized expressions involving Minkowski tensors and vectors,
unfolding multiplicative structures into expanded sums for subsequent simplification.
Does not expand into explicit components but rather expands nested factorizations.

**Factorization Expansion:**
- `A(μ) * (B(ν) + C(ν)) → A(μ)*B(ν) + A(μ)*C(ν)`
- Nested products with Minkowski indices get distributed
- Parenthesized expressions are expanded algebraically
- Prepares expressions for metric simplification and contractions

**Applications:**
- Expanding factorized tensor expressions before simplification
- Preparing for index contraction algorithms
- Unfolding nested products in field theory calculations
- Algebraic manipulation of relativistic expressions

# Arguments
- `self_`: expression containing factorized terms with Minkowski indices

# Returns
Expanded expression with factorizations unfolded.
```

```
# Examples:
```python
import symbolica as sp
from symbolica.community.idenso import expand_mink

# Expand factorized vector expression
p = sp.S('p')
q = sp.S('q')
r = sp.S('r')
mu = sp.S('mu')
factorized = p(mu) * (q(mu) + r(mu))
expanded = expand_mink(factorized)  # p(mu)*q(mu) + p(mu)*r(mu)

# Complex factorization
A = sp.S('A')
expr = A * (p(mu) * q(mu) + r(mu))
result = expand_mink(expr)  # A*p(mu)*q(mu) + A*r(mu)
```
```

```
def expand_mink(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`expand_mink_bis`

```
Expands factorized terms containing both Minkowski and bispinor indices.
```


```
Expands factorized terms containing both Minkowski and bispinor indices.
```


```
Combines the functionality of `expand_mink()` and `expand_bis()` to perform
simultaneous expansion of factorized expressions involving both spacetime
and spinor indices.

# Arguments
- `self_`: expression containing factorized terms with both index types

# Returns
Expanded expression with all factorizations unfolded.
```

```
def expand_mink_bis(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`initialize`

```
Register Idenso's built-in representations and algebra symbols with Symbolica.
```


```
Register Idenso's built-in representations and algebra symbols with Symbolica.
```


```
Symbolica calls this during community-module initialization. Calling it again
is safe and ensures the standard Lorentz, spinor, and color objects exist.
```

```
def initialize() -> None:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

list_dangling

Lists the dangling (external, uncontracted) indices present in the expression.

Lists the dangling (external, uncontracted) indices present in the expression.

Identifies and returns all indices that are not summed over (i.e., not dummy
indices). These are the "free" indices that appear in the final result and
determine the tensor rank of the expression. For dualizable representations,
downstairs indices are represented wrapped in `dind(...)`.

This is essential for:
- Verifying index conservation in tensor equations
- Determining the rank and structure of tensor expressions
- Debugging index contractions

Arguments
- `self_`: tensor expression to analyze

Returns
A list of expressions, each representing a free (dangling) index.

Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import (
    list_dangling,
)

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, nu, x)  # T(mu, nu; x)
# print(tensor_with_args)
print(list_dangling(tensor_with_args.to_expression()))
```

```
def list_dangling(self_: Expression) -> list[Expression]:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

simplify_color

Applies SU(N) color algebra rules to simplify color group structures.

Applies SU(N) color algebra rules to simplify color group structures.

Performs comprehensive simplifications of SU(N) color algebra including:

Structure constants:
- `f^{abc}f^{ade} = CA δ^{bc}` (Casimir relations)
- `f^{abc}f^{bcd} = CA f^{acd}` (Jacobi identities)
- Antisymmetry: `f^{abc} = -f^{bac}`

Generators and traces:

```
- `Tr(T^a T^b) = TR δ^{ab}` (orthogonality)
- `T^a_{ij} T^a_{kl} = δ_{il}δ_{jk}/Nc - δ_{ij}δ_{kl}/Nc²` (Fierz identity)
- `(T^a)² = CF` (fundamental Casimir)

**Color factors:**
- `Nc`: Number of colors
- `CA = Nc`: Adjoint Casimir
- `CF = (Nc² - 1)/(2Nc)`: Fundamental Casimir
- `TR = 1/2`: Normalization factor

# Arguments
- `self_`: expression containing SU(N) color structures

# Returns
Simplified expression with color algebra reduced to scalar factors (`Nc`, `CA`, `CF`,
`TR`) when possible.

# Notes
If explicit color indices remain after simplification, it indicates the expression
could not be fully reduced to color-scalar form.

def simplify_color(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

simplify_gamma

```
Applies Clifford algebra rules and trace identities to simplify gamma matrix
expressions.

Applies Clifford algebra rules and trace identities to simplify gamma matrix
expressions.

Performs comprehensive simplifications of Dirac gamma matrix algebra including:
- **Anticommutation relations**: `{γᵘ, γᵛ} = 2gᵘᵛ`
- **Trace identities**: `Tr(γᵘ) = 0`, `Tr(γᵘγᵛ) = 4gᵘᵛ`, etc.
- **Gamma5 properties**: `{γ₅, γᵘ} = 0`, `(γ₅)² = 1`
- **Chain simplifications**: Reduces products of gamma matrices
- **Contraction rules**: Simplifies contracted gamma matrix products

The function recognizes gamma matrices represented as
`spenso::gamma(spenso::mink(dim,mu), spenso::bis(dim,alpha), spenso::bis(dim,beta))`
where `mu` is the Lorentz index and `alpha`, `beta` are spinor indices.
These can be easily created using the hep_lib.

# Arguments
- `self_`: expression containing gamma matrix products and traces

# Returns
The simplified expression with gamma algebra applied.

# Examples:
```python
from symbolica.community.spenso import TensorLibrary, TensorName
from symbolica.community.idenso import simplify_gamma
from symbolica import S, Expression
# Get HEP library with standard tensors
```
```

```
hep_lib = TensorLibrary.hep_lib()
# Access standard tensors like gamma matrices
gamma_structure = hep_lib[S("spenso::gamma")]
print(gamma_structure)
print(simplify_gamma(gamma_structure(7, 3, 4) * gamma_structure(3, 7, 4)))
```

```
def simplify_gamma(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`simplify_metrics`

Simplifies contractions involving metric tensors and identity tensors.

Simplifies contractions involving metric tensors and identity tensors.

Applies fundamental tensor algebra rules for metric and identity tensors:

Metric tensor rules:
- `gᵘᵛ pᵥ → pᵘ` (index raising/lowering)
- `gᵘᵛ gᵥρ → gᵘρ` or `δᵘρ` (metric composition)
- `gᵘᵤ → D` (dimension of spacetime)
- `ηᵘᵛ pᵥ → pᵘ` (flat metric contractions)

Identity tensor rules:
- `δᵘᵛ pᵥ → pᵘ` (Kronecker delta contraction)
- `δᵘᵤ → D` (trace of identity)

The function recognizes metrics as `spenso::g(...)`

Arguments
- `self_`: expression containing metric/identity tensor contractions

Returns
The simplified expression with metric rules applied.

Examples:
```python
from symbolica.community.idenso import simplify_metrics, to_dots
from symbolica.community.spenso import Representation, TensorName
q = TensorName("q")
g = TensorName.g()
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
print(simplify_metrics(g(mu, nu) * q(mu)))
```

```
def simplify_metrics(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`to_dots`

Converts contracted Lorentz/Minkowski indices into dot product notation.

Converts contracted Lorentz/Minkowski indices into dot product notation.

Automatically identifies and converts patterns like `p(mink(D, mu)) * q(mink(D, mu))`
into the compact, representation-carrying `dot(p(mink(D)), q(mink(D)))` notation.
This
simplification is essential for physics calculations involving four-vectors.

The function recognizes:
- Contracted vector indices: `pᵘqᵤ → p·q`
- Multiple contractions: `pᵘqᵤrᵛsᵥ → (p·q)(r·s)`
- Self-contractions: `pᵘpᵤ → p²`

Arguments
- `self_`: expression containing contracted Minkowski vector indices

Returns
The expression with vector contractions converted to dot products.

Examples:
```python
from symbolica.community.idenso import to_dots
from symbolica.community.spenso import Representation, TensorName
p = TensorName("p")
q = TensorName("q")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")

print(to_dots( p(mu)*q(mu)))
```

```
def to_dots(self_: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`wrap_dummies`

Wraps only the dummy (contracted) indices within the expression using a header
symbol.

Wraps only the dummy (contracted) indices within the expression using a header
symbol.

Similar to `wrap_indices`, but selectively identifies and wraps only contracted
indices (those appearing once upstairs and once downstairs, or twice in a
self-dual representation), leaving external (dangling) indices untouched.
This is crucial for proper index management in tensor calculations.

Contracted indices are those that:
- Appear in both upper and lower positions (for dualizable reps)
- Appear twice in the same position (for self-dual reps)
- Are summed over (Einstein summation convention)

Arguments
- `self_`: input expression containing both dummy and free indices
- `header`: symbol to use as wrapper function name for dummy indices only

```
# Returns
A new expression with only contracted indices wrapped.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import simplify_metrics, wrap_dummies

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, nu, x)  # T(mu, nu; x)
# print(tensor_with_args)
print(wrap_dummies(tensor_with_args.to_expression(), sp.S("wrap")))

```
```

```
def wrap_dummies(self_: Expression, header: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

`wrap_indices`

Wrap all abstract indices with a header symbol # Arguments - `self_`: input expression containing tensor indices - `header`: symbol to use as the wrapper function for all indices # Returns Expression with all indices wrapped by the header symbol.

```
Wrap all abstract indices with a header symbol

# Arguments
- `self_`: input expression containing tensor indices
- `header`: symbol to use as the wrapper function for all indices

# Returns
Expression with all indices wrapped by the header symbol.

# Examples:
```python
from symbolica.community.spenso import TensorName, Slot, Representation
import symbolica as sp
from symbolica.community.idenso import wrap_indices

T = TensorName("T")
rep = Representation.euc(3)
# With slots (creates TensorIndices)
mu = rep("mu")
nu = rep("nu")
x = sp.S("x")
tensor_with_args = T(mu, nu, x)  # T(mu, nu; x)
print(tensor_with_args)
print(wrap_indices(tensor_with_args.to_expression(), sp.S("wrap")))
```
```

```
def wrap_indices(self_: Expression, header: Expression) -> Expression:
```

Requires features: `python, python_stubgen`

Exhaustive reference · Source

Version information

Save these identifiers with published results and include them in issue reports so that collaborators can reproduce the same software environment.

- **Documentation channel:** `latest`
- **Documentation route:** `/products/idenso/latest/`
- **Source revision:** `cbfc3a5827679de14e9373edd6b332e6b9b28a55`

Package versions and enabled features

- `gammaloop/gammalooprs` — `0.3.4` (`no optional features`)
- `gammaloop/gammaloop-api` — `0.1.0` (`features: cli`)
- `gammaloop/gammaloop` — `0.3.4` (`features: python_api, python_abi, python_stubgen`)
- `linnet/linnet` — `0.17.0` (`features: nodestore-vec, serde, bincode, drawing, symbolica`)
- `linnet/linnet-py` — `0.1.0` (`features: extension-module, abi3-py310`)
- `spenso/spenso` — `0.6.0` (`features: shadowing`)
- `spenso/spenso-macros` — `0.3.2` (`features: shadowing`)
- `spenso/spenso-hep-lib` — `0.1.7` (`no optional features`)
- `spenso/spynso3` — `0.1.2` (`features: python_stubgen`)
- `idenso/idenso` — `0.3.0` (`features: bincode, reference-cases`)
- `idenso/idenso` — `0.3.0` (`features: python, python_stubgen`)
- `vakint/vakint` — `0.1.2` (`no optional features`)
- `vakint/vakint` — `0.1.2` (`features: symbolica_community_module, python_stubgen`)